ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 7. МЕТОДЫ РАЗБИЕНИЯ ДАННЫХ В СИСТЕМАХ ОБНАРУЖЕНИЯ КИБЕРАТАК

План лекции:

1. Формирование выборок для задач кибербезопасности.
2. Схемы разделения данных для разных режимов обучения.
3. Способы разбиения данных.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
pd.set_option('display.max_columns', None)
```

Методы разбиения данных в системах обнаружения кибератак

1. Формирование выборок для задач кибербезопасности

Валидация моделей - это процесс проверки того, насколько хорошо модель (статистическая, ML-модель, и т. п.) отражает реальность и пригодна для решения поставленной задачи. Проще говоря: валидация отвечает на вопрос «работает ли модель в условиях, близких к реальным, и можно ли ей доверять?».

Причины использования валидации моделей выявления киберугроз.

1. Обучение модели и проверка её способности к обобщению.

Цель обучения модели состоит не в том, чтобы максимально хорошо описать уже имеющиеся данные. Модель должна правильно работать с новыми событиями, которые не участвовали в её построении.

Если модель обучается и проверяется на одних и тех же данных, полученное качество характеризует способность модели запоминать наблюдения, а не обнаруживать новые атаки.

2. Связанные события в данных кибербезопасности.

Один реальный процесс может создавать множество связанных строк:

- одно сетевое соединение состоит из нескольких пакетов;
- одна пользовательская сессия создаёт последовательность событий;
- один процесс создаёт несколько записей в журнале;
- одно сканирование портов создаёт сотни сетевых соединений;
- один инцидент состоит из нескольких этапов;
- одна атакующая кампания содержит несколько похожих инцидентов.

Например, одна атака сканирования создала 1000 событий. При случайном разбиении:

- 800 событий могут попасть в обучение;
- 200 событий — в тестирование.

При этом с большой вероятностью значительная часть этих событий попадёт и в обучение, и в тест. Хотя события различаются, тестовая часть содержит практически ту же атаку, которую модель уже видела.

Модель «запоминает» паттерн атаки на обучении, а потом показывает отличную метрику на тесте, потому что тест содержит практически ту же атаку. Это не значит, что модель научилась обобщать — она просто «узнаёт» знакомый шаблон. Полученный результат оценивания качества модели будет завышен.

Поэтому необходимо учитывать не только пересечение отдельных строк, но и пересечение групп.

3. Временная зависимость.

События кибербезопасности имеют временной порядок. При реальной эксплуатации модель обучается на исторических данных и применяется к будущим событиям.

Случайное перемешивание наблюдений способно создать обратную ситуацию: будущие события попадут в обучение, а исторические — в тестирование.

4. Изменение нормального поведения.

Распределение данных в кибербезопасности не является постоянным. Оно изменяется из-за:

- увеличения числа пользователей;
- появления новых серверов;
- внедрения облачных сервисов;
- обновления программного обеспечения;
- изменения сетевой архитектуры;
- изменение режима работы сотрудников;
- изменения расписания резервного копирования.

Это явление называется **дрейфом данных**.

Для его отслежвания нужны методы адаптивной валидации.

Исходя из описанных причин для корректной разработки модели выявления кибератак данные необходимо разделить как минимум на три части:

$$D = \text{train} \cup \text{validation} \cup \text{test}.$$

Для моделей выявления аномалий часто требуется дополнительная калибровочная выборка:

$$D = \text{train} \cup \text{validation} \cup \text{calibration} \cup \text{test}.$$

Разделение должно быть произведено до обучения модели и предварительной обработки данных.

Обучающая выборка (train) используется непосредственно для построения модели. На ней оцениваются первоначальные параметры алгоритма.

Обучающая выборка должна:

- соответствовать выбранному режиму обучения;
- содержать достаточное количество событий;
- включать основные режимы поведения;
- не содержать объекты из финального теста;
- не пересекаться с validation по сессиям или инцидентам;
- не включать информацию из будущего;
- использоваться для обучения всех преобразований данных.

Валидационная выборка (validation) используется для подбора гиперпараметров (не параметров модели, а настроек алгоритма): `n_neighbors` у LOF, число бинов у HBOS и т. п.

Варианты гиперпараметров перебираются, для каждого набора модель обучается на train и оценивается качество на validation. Выбирают набор гиперпараметров с лучшей обобщающей способностью.

Важное правило: validation не должна использоваться для финальной оценки качества. Иначе оценка будет оптимистичной.

Валидационная выборка:

- не участвует в обучении модели;
- может многократно использоваться при сравнении конфигураций гиперпараметров;
- влияет на выбор итоговой модели;
- не является независимым финальным тестом;
- должна воспроизводить условия будущей эксплуатации;
- не должна пересекаться с train по сессиям, инцидентам или временным окнам.

После многократного сравнения моделей разработчик косвенно подстраивается под validation. Поэтому показатели на ней могут быть оптимистичными.

Калибровочная выборка calibration нужна для выбора порога, преобразующего непрерывную оценку аномальности события в бинарное решение.

Это отдельная, часто специфическая выборка, которая особенно важна в задачах детекции аномалий.

Тестовая выборка test используется для однократной независимой проверки настроенной модели.

Перед применением теста должны быть зафиксированы:

- алгоритм;
- набор признаков;

- способ предварительной обработки;
- гиперпараметры;
- порог формирования тревоги.

Тестовая выборка должна:

- не использоваться для обучения;
- не использоваться для выбора гиперпараметров;
- содержать новые события;
- воспроизводить рабочую долю нормы и атак;
- относиться к будущему периоду, если модель будет работать с потоком.

Причины формирования отдельной калибровочной выборки.

На первый взгляд можно использовать одну validation-выборку и для выбора гиперпараметров, и для определения порога. Технически это возможно, но имеет несколько недостатков.

1. validation уже использовалась для выбора модели.

Предположим, проверялось 20 конфигураций. Выбрана та, которая лучше всего сработала именно на текущей validation.

Эта модель может случайно особенно хорошо соответствовать особенностям validation. Если по тем же данным выбрать порог, происходит повторная подстройка к одному набору.

Чем больше решений принимается по одной выборке, тем выше риск переобучения на неё.

2. После выбора модели validation можно добавить в обучение.

После выбора гиперпараметров полезно увеличить объём нормальных обучающих данных, добавив к ним валидационную выборку.

В этом случае validation становится частью обучения окончательной модели. Использовать её оценки для порога уже нельзя: модель видела эти объекты.

Поэтому нужна отдельная калибровочная выборка, которая не участвоваа в повторном обучении.

3. У validation и calibration разные задачи.

validation выбирает относительного победителя:

Какая из моделей лучше?

calibration выбирает рабочую границу:

Какой score считать тревогой?

Это разные статистические задачи, поэтому желательно использовать разные данные.

2. Схемы разделения данных для разных режимов обучения

Состав выборок и способ использования меток атак зависят от режима обучения.

1. Обучение с учителем.

Общая схема:

```
train
├── нормальные события с метками
└── атаки с метками

validation
├── нормальные события с метками
└── атаки с метками

test
├── новые нормальные события с метками
├── новые экземпляры известных атак
└── ранее не использовавшиеся атаки
```

Последовательность действий:

```
train
  ↓
Обучение моделей с использованием меток
  ↓
validation
  ↓
Выбор гиперпараметров модели
  ↓
calibration
  ↓
Выбор порога
  ↓
test
  ↓
Независимая проверка
```

Обучение без учителя.

Общая схема:

```
train
├── нормальные события без использования меток
└── неизвестные атаки без использования меток

validation
├── нормальные события
└── атаки или неизвестные аномалии

calibration
├── нормальные события
└── возможные атаки — в зависимости от способа выбора порога

test
├── новые нормальные события
└── новые атаки
```

Последовательность действий:

```
train
  ↓
Обучение модели без меток
  ↓
validation
  ↓
Выбор гиперпараметров модели
  ↓
calibration
  ↓
Выбор порога
  ↓
test
  ↓
Независимая проверка
```

Обучении только на нормальных данных.

Общая схема:

```
train
└── только подтверждённые нормальные события

validation
├── отложенные нормальные события
└── при наличии — известные атаки

calibration
└── только новые подтверждённые нормальные события

test
├── новые нормальные события
├── известные атаки
```

```
└── ранее не использовавшиеся атаки
```

Последовательность действий:

```
train
  ↓
Обучение только на нормальных событиях
  ↓
validation
  ↓
Выбор гиперпараметров модели
  ↓
calibration
  ↓
Порог по новым нормальным событиям
  ↓
test
  ↓
Проверка на новых нормальных событиях и атаках
```

3. Подготовка исходных данных

In [ ]:
```
df = pd.read_csv("/content/cybersecurity_validation_dataset.csv",
                 parse_dates=["timestamp",
                              "window_start",
                              "window_end"])
df.head()
```

Out[ ]:

	event_id	timestamp	window_start	window_end	host_id	host_role	user_id	sessi
0	event-000001	2026-01-01 00:16:27	2026-01-01 00:11:27	2026-01-01 00:16:27	srv-04	server	user-051	no ses
1	event-000002	2026-01-01 00:17:27	2026-01-01 00:12:27	2026-01-01 00:17:27	srv-04	server	user-051	no ses
2	event-000003	2026-01-01 00:18:27	2026-01-01 00:13:27	2026-01-01 00:18:27	srv-04	server	user-051	no ses
3	event-000004	2026-01-01 00:19:27	2026-01-01 00:14:27	2026-01-01 00:19:27	srv-04	server	user-051	no ses
4	event-000005	2026-01-01 02:28:19	2026-01-01 02:23:19	2026-01-01 02:28:19	ws-038	workstation	user-032	no ses

Назначение столбцов учебного датасета.

`event_id` : идентификатор события.

Временные признаки:

Каждая строка описывает активность, собранную за определённый временной интервал (5 минут).

- `timestamp` : время регистрации события (совпадает с концом временного окна `window_end`);
- `window_start` : содержит время начала временного интервала, по которому рассчитаны признаки;
- `window_end` : содержит время окончания временного интервала, по которому рассчитаны признаки.

Устройства, пользователи и группы:

- `host_id` :— идентификатор устройства (префиксы: `ws` — рабочая станция, `srv` — сервер);
- `host_role` : роль устройства (возможные значения: `workstation` и `server`):
- `user_id` : идентификатор пользователя (префиксы: `user` — пользовательская учётная запись, `svc` — сервисная учётная запись);
- `session_id` : идентификатор сессии (префиксы: `normal` , `attack`). Одна сессия содержит несколько связанных событий;
- `incident_id` : идентификатор киберинцидента. Все события одной атаки получают одинаковый `incident_id` ;
- `campaign_id` : идентификатор атакующей кампании;
- `validation_group` : универсальная группа для валидации. Это специально созданный служебный столбец.

`validation_group` — это служебный идентификатор группы взаимосвязанных событий, которые при разделении датасета нельзя разрывать между обучающей и проверочной выборками.

Проще говоря, этот столбец сообщает алгоритму разбиения: «эти строки относятся к одной и той же активности. Помести их целиком либо в `train` , либо в `validation»` .

Самой модели столбец `validation_group` не передаётся. Он нужен только для правильного разделения данных.

Почему нельзя разделять отдельные строки случайно?

В датасете одна строка соответствует отдельному событию, но несколько строк могут описывать одну и ту же активность.

Например, одна нормальная пользовательская сессия содержит пять событий:

event_id	session_id	is_attack
event-000101	normal-session-0025	0
event-000102	normal-session-0025	0
event-000103	normal-session-0025	0
event-000104	normal-session-0025	0
event-000105	normal-session-0025	0

Эти события:

- созданы одним пользователем;
- связаны с одним устройством;
- зарегистрированы примерно в одно время;
- имеют похожие значения признаков.

Если выполнить обычное случайное разбиение строк, получится, например:

Событие	Часть данных
event-000101	train
event-000102	train
event-000103	validation
event-000104	train
event-000105	validation

Тогда модель обучается на нескольких событиях сессии, а проверяется на других событиях той же сессии.

Такая проверка слишком простая: в `validation` оказываются события, почти совпадающие с теми, которые модель уже видела в `train`. Полученная оценка качества может оказаться завышенной.

Формирование столбца `validation_group`.

Для нормальных событий `validation_group = session_id`, так как для них дентификатор инцидента `incident_id` отсутствует.

Для аномальных событий `validation_group = incident_id`. Потому что теоретически один инцидент может состоять из нескольких сессий:

```
inc-lateral-movement-05
├── session-01: первоначальный вход
├── session-02: подключение к серверу
└── session-03: перемещение на другой хост

Если группировать только по session_id, разные сессии одного
инцидента могут попасть в разные части:
```

```
session-01 → train
session-02 → validation
session-03 → validation
```

Контекст сетевой активности:

- `traffic_type` : тип сетевой активности.
- `connections_per_min` : количество соединений в минуту;
- `unique_destinations` : число уникальных адресатов;
- `outbound_mb` : объём исходящего трафика;
- `failed_logins` : число неудачных попыток входа;
- `duration_sec` : продолжительность активности.

Истинные метки событий:

- `scenario` : сценарий события;
- `is_attack` : бинарная истинная метка (0 — нормальное событие, 1 — атака).

Служебное разбиение набора данных:

- `dataset_part` : части датасета (возможные значения: `development` и `final_test`).

Производные временные столбцы:

- `day_index` : номер дня наблюдения (в датасете 0 соответствует первому дню наблюдения, 119 — последнему);
- `hour` : час суток;
- `day_of_week` : день недели.

Итоговое разделение столбцов.

In [ ]:
```
# Числовые поведенческие характеристики.
MODEL_FEATURES = ["connections_per_min", "unique_destinations",
                  "outbound_mb", "failed_logins", "duration_sec"]
```

In [ ]:
```
# Целевая метка
TARGET = "is_attack"
```

In [ ]:
```
# Временные столбцы
TIME_COLUMNS = ["timestamp", "window_start", "window_end",
                "day_index", "hour", "day_of_week"]
```

In [ ]:
```
# Группировочные столбцы
GROUP_COLUMNS = ["host_id", "user_id", "session_id",
                 "incident_id", "campaign_id", "validation_group"]
```

Контекст:

- числовые поведенческие характеристики используются для построения модели;

- временные и групповые столбцы — для корректного разбиения;
- `scenario` и `is_attack` — для формирования учебного эксперимента и проверки качества моделей.

4. Способы разбиения данных

Существует достаточно большое количество подходов к разбиению данных, которые можно разделить на группы подобных способов.

1. Однократное и повторное случайное разбиение:

- Hold-out, стратифицированный Hold-out;
- ShuffleSplit, StratifiedShuffleSplit;
- Bootstrap.

2. Кросс-валидация по блокам:

- KFold, StratifiedKFold, RepeatedKFold, RepeatedStratifiedKFold;
- LeaveOneOut, LeavePOut.

3. Групповые способы:

- GroupShuffleSplit, GroupKFold;
- StratifiedGroupKFold;
- LeaveOneGroupOut, Leave-One-Attack-Type-Out.

4. Временные способы:

- временной Hold-out;
- блочная временная валидация;
- TimeSeriesSplit;
- скользящее окно;
- gap;
- Purged CV.

4.1. Однократное и повторное случайное разбиение

При случайном разбиении объекты перемешиваются и распределяются случайным образом между всеми необходимыми выборками:

`train → validation → calibration → test`.

Предполагается, что отдельные строки датасета являются независимыми и извлечены из одного распределения.

Случайное разбиение хорошо работает, если:

- события независимы;

- отсутствует выраженный временной порядок;
- строки не объединены в сессии и инциденты;
- поведение системы не изменяется со временем;
- требуется проверить работу на новых объектах из того же распределения.

Для данных кибербезопасности эти условия часто нарушаются. Один пользователь, компьютер, сеанс или инцидент может создавать множество зависимых строк.

В наборе данных уже создано верхнеуровневое временное разделение. Техническое значение `dataset_part = development` обозначает строки, которые далее будут распределяться между `train`, `validation` и `calibration`.

In [ ]:
```
# Данные для разбиения
source_df = (df[df["dataset_part"] == "development"].copy().reset_index(drop=Tru

# Тестовые данные
test = (df[df["dataset_part"] == "final_test"].copy().reset_index(drop=True))

print("Данных для разбиения:", len(source_df))
print("Данных для тестирования:", len(test))
```

```
Данных для разбиения: 5168
Данных для тестирования: 1137
```

Вспомогательные функции проверки разбиения.

Функция `describe_part` возвращает основные характеристики одной части датасета.

In [ ]:
```
# @title
def describe_part(name, data):

    return {
        "part": name,
        "events": len(data),
        "normal": int(
            (data["is_attack"] == 0).sum()
        ),
        "attacks": int(
            (data["is_attack"] == 1).sum()
        ),
        "attack_fraction": (
            data["is_attack"].mean()
        ),
        "groups": (
            data["validation_group"].nunique()
        ),
        "time_min": data["timestamp"].min(),
        "time_max": data["timestamp"].max()
    }

def split_summary(
    train,
    validation,
    calibration,
    test
```

```
):
    """
    Формирует таблицу характеристик
    всех четырёх частей.
    """

    summary = pd.DataFrame(
        [
            describe_part("train", train),
            describe_part(
                "validation",
                validation
            ),
            describe_part(
                "calibration",
                calibration
            ),
            describe_part("test", test)
        ]
    )

    summary["attack_percent"] = (
        summary["attack_fraction"] * 100
    )

    return summary.drop(
        columns="attack_fraction"
    )
```

Функция `count_group_overlap` считает количество групп, события которых находятся одновременно в двух частях - `train` и `validation`.

Функция `print_overlap_report` выводит отчёт о пересечении групп и инцидентов между тремя выборками: `train`, `validation` и `calibration`. Она использует функцию `count_group_overlap`: та считает пересечения, а `print_overlap_report` выводит результаты с подписями.

In [ ]:
```
# @title
def count_group_overlap(
    first,
    second,
    group_column="validation_group"
):

    first_groups = set(
        first[group_column].dropna()
    )

    second_groups = set(
        second[group_column].dropna()
    )

    return len(
        first_groups.intersection(
            second_groups
        )
    )
```

```python
def print_overlap_report(
    train,
    validation,
    calibration
):
    """
    Показывает пересечения связанных событий.
    """

    print("Пересечение validation_group:")

    print(
        "train и validation:",
        count_group_overlap(
            train,
            validation
        )
    )

    print(
        "train и calibration:",
        count_group_overlap(
            train,
            calibration
        )
    )

    print(
        "validation и calibration:",
        count_group_overlap(
            validation,
            calibration
        )
    )


    print("\nПересечение incident_id:")

    print(
        "train и validation:",
        count_group_overlap(
            train,
            validation,
            group_column="incident_id"
        )
    )

    print(
        "train и calibration:",
        count_group_overlap(
            train,
            calibration,
            group_column="incident_id"
        )
    )

    print(
        "validation и calibration:",
        count_group_overlap(
            validation,
```

```
            calibration,
            group_column="incident_id"
        )
    )
```

Однократное случайное разбиение Hold-out

Hold-out - однократное случайное разбиение данных на обучающую и тестовую выборки. Один раз случайно перемешивает данные и делит их на части.

Для реализации этого метода используется функция `train_test_split` модуля `model_selection` библиотеки `sklearn`. Она случайно перемешивает объекты и возвращает две части.

Основные параметры:

- `test_size`: доля (или число объектов) в тестовой выборке (`test`);
- `train_size`: доля/число в обучающей выборке (`train`). Если задан, `test_size` вычисляется как остаток. Обычно используют только test_size;
- `random_state`: фиксирует случайность. В ИБ это обязательно: воспроизводимость экспериментов нужна для отчётов и сравнения методов;
- `shuffle` (по умолчанию `True`): перемешивать ли данные перед разбиением. Для временных данных (трафик, сессии) лучше ставить `False` и делать свой `time-based split`, иначе будет утечка из будущего;
- `stratify`: сохраняет пропорции классов в `train/test`. В ИБ полезно, если есть разметка (норма/атака или типы атак).

Для получения трех выборок `train`, `validation` и `calibration` эту функцию нужно вызвать дважды.

Выберем пропорции:

- train — 60%
- validation — 20%
- calibration — 20%

In [ ]:
```
# Первый этап:
# 20% объектов отправляем в calibration.
remaining, calibration = train_test_split(source_df,
                                          test_size = 0.20,
                                          shuffle = True,
                                          random_state = 42)
# Второй этап:
# Чтобы обеспечить 20% валидационных данных от исходных данных
# нужно взыть 25% от remaining.
train, validation = train_test_split(remaining,
                                     test_size = 0.25,
                                     shuffle = True,
                                     random_state = 42)
# Для удобства сбрасываем индексы.
train = train.reset_index(drop = True)
```

```
validation = validation.reset_index(drop = True)
calibration = calibration.reset_index(drop = True)
```

Результаты разделения данных.

In [ ]:
```
random_summary = split_summary(train = train,
                               validation = validation,
                               calibration = calibration,
                               test = test)
random_summary
```

Out[ ]:

	part	events	normal	attacks	groups	time_min	time_max	attack_percent
0	train	3100	2953	147	1001	2026-01-01 00:16:27	2026-04-10 23:47:44	4.741935
1	validation	1034	973	61	683	2026-01-01 00:17:27	2026-04-10 23:45:44	5.899420
2	calibration	1034	978	56	680	2026-01-01 02:32:19	2026-04-10 23:46:44	5.415861
3	test	1137	1076	61	227	2026-04-11 01:45:36	2026-04-30 22:29:24	5.364996

Проблемы использования случайного Hold-out для связанных событий.

1. Пересечение групп однотипных событий.

In [ ]:
```
print_overlap_report(train, validation, calibration)
```

Пересечения `validation_groups` показывают, сколько значений столбца `validation_group` одновременно присутствует в разных выборках (например в `train` и `validation`).

`validation_group` — это техническая единица разбиения. Она объединяет события, которые желательно не разделять между выборками, например:

- события одного узла за определённый период;
- сетевые события одной сессии;
- связанные наблюдения из одного временного окна;
- события одной атаки вместе с сопутствующим фоновым трафиком.

Пример:

- `train`: `VG_01`, `VG_02`, `VG_03`
- `validation`: `VG_03`, `VG_04`

Общая группа — `VG_03`, поэтому для этого случая:

```
Пересечение validation_group:
train и validation: 1
```

Это означает, что часть одной логически связанной группы оказалась в `train`, а другая часть — в `validation`.

Пересечения `incident_id` показывают, сколько конкретных инцидентов, определяемых по `incident_id`, одновременно присутствует в `train` и `validation`.

Пример:

- `train`: `INC_01`, `INC_02`, `INC_03`
- `validation`: `INC_03`, `INC_04`

Общий инцидент — `INC_03`, поэтому для этого случая:

```
Пересечение incident_id:
train и validation: 1
```

Это означает, что события одного инцидента были разделены между обучением и проверкой. Модель может частично «увидеть» этот инцидент во время обучения, что создаёт утечку данных и завышает оценку качества.

При подсчёте следует учитывать только настоящие идентификаторы инцидентов. Значения `NaN`, `normal` или `no_incident`, обозначающие отсутствие инцидента, считать общим инцидентом не нужно.

Один `validation_group` может содержать несколько инцидентов, а один инцидент теоретически может продолжаться в нескольких группах.

Например:

Событие	`validation_group`	`incident_id`
1	`VG_01`	`INC_10`
2	`VG_01`	`INC_10`
3	`VG_01`	`INC_11`
4	`VG_02`	`INC_11`

Если `VG_01` попадёт в `train`, а `VG_02` — в `validation`, то:

```
Пересечение validation_groups = 0
Пересечение incident_id = 1
```

Группы не пересекаются, но инцидент `INC_11` всё равно разделён между выборками.

Поэтому:

- Пересечение `validation_groups = 0` подтверждает корректное разделение выбранных групп;
- Пересечение `incident_id = 0` подтверждает отсутствие разделённых между выборками инцидентов;

Для наиболее строгой проверки желательно получить нулевые значения обоих показателей.

Таким образом, для наших данных после разделения события одних и тех же групп и одних и тех же инцидентов оказались одновременно в разных частях.

Например:

```
inc-port_scan-03
├── часть событий → train
└── часть событий → validation
```

In [ ]:
```
train[train["validation_group"] == 'inc-port_scan-03']
```

Out[ ]:

	event_id	timestamp	window_start	window_end	host_id	host_role	user_id	s
1946	event-002043	2026-02-10 11:08:00	2026-02-10 11:03:00	2026-02-10 11:08:00	ws-033	workstation	user-057	p
2016	event-002045	2026-02-10 11:12:00	2026-02-10 11:07:00	2026-02-10 11:12:00	ws-033	workstation	user-057	p
2288	event-002048	2026-02-10 11:18:00	2026-02-10 11:13:00	2026-02-10 11:18:00	ws-033	workstation	user-057	p
2856	event-002044	2026-02-10 11:10:00	2026-02-10 11:05:00	2026-02-10 11:10:00	ws-033	workstation	user-057	p

In [ ]:
```
validation[validation["validation_group"] == 'inc-port_scan-03']
```

Out[ ]:

	event_id	timestamp	window_start	window_end	host_id	host_role	user_id	se
190	event-002047	2026-02-10 11:16:00	2026-02-10 11:11:00	2026-02-10 11:16:00	ws-033	workstation	user-057	po

Модель может обучиться на нескольких событиях сканирования, а затем проверяться на практически таких же событиях того же инцидента.

Следовательно отсутствие одинаковых строк ещё не означает отсутствие утечки данных.

Проблема устраняется с помощью групповых способов разбиения.

2. Пересечение временных периодов.

In [ ]:
```
random_summary[["part","time_min","time_max"]]
```

Out[ ]:

	part	time_min	time_max
0	train	2026-01-01 00:16:27	2026-04-10 23:47:44
1	validation	2026-01-01 00:17:27	2026-04-10 23:45:44
2	calibration	2026-01-01 02:32:19	2026-04-10 23:46:44
3	test	2026-04-11 01:45:36	2026-04-30 22:29:24

Выборки `train`, `validation` и `calibration` охватывают практически один и тот же период: `01.01.2026–10.04.2026`.

При случайном разбиении Hold-out события за январь–апрель перемешиваются: в каждой выборке оказываются данные и за январь, и за апрель. Модель проверяется на других событиях того же периода, хотя часть более поздних событий уже использовалась при обучении. Но на практике модель нужно обучать на накопленных данных, а применяют к будущим событиям.

Эта проблема устраняется использованием временной валидации, которая воспроизводит эту последовательность: например, январь–февраль — обучение, март — валидация, начало апреля — калибровка. Так можно проверить, как модель работает на более поздних данных, когда характер сетевой активности и атак мог измениться.

Ограничения метода Hold-out.

- Результат сильно зависит от одного случайного разбиения.
- При малом числе атак отдельные типы атак могут не попасть в `validation` или `calibration`.
- Часть данных исключается из обучения.
- Случайное разбиение не предотвращает утечку между связанными событиями одной атаки, хоста или пользователя.
- Не учитывает временной порядок событий.

Стратифицированный Hold-out

Стратификация стремится сохранить долю классов исходной выборки в каждой части `train`, `validation` и `calibration`.

Для этого целевая переменная передаётся в параметр: `stratify`.

In [ ]:
```
remaining, calibration = train_test_split(source_df,
                                          test_size = 0.20,
                                          shuffle = True,
                                          random_state = 42,
                                          # Сохраняем долю нормы и атак.
                                          stratify = source_df["is_attack"])

train, validation = train_test_split(remaining,
                                     test_size = 0.25,
                                     shuffle = True,
                                     random_state = 42,
                                     # На втором этапе стратификация
                                     # выполняется по оставшимся объектам.
                                     stratify = remaining["is_attack"])

train = train.reset_index(drop = True)
validation = validation.reset_index(drop = True)
calibration = calibration.reset_index(drop = True)
```

Доля атак в исходной выборке.

In [ ]:
```
# Доля атак в данных для разбиения
print(f'Доля атак в исходной выборке: {np.round(source_df['is_attack'].mean()*10
```

```
Доля атак в исходной выборке: 5.11%
```

In [ ]:
```
stratified_summary = split_summary(train = train,
                                   validation = validation,
                                   calibration = calibration,
                                   test = test)
stratified_summary
```

Out[ ]:

	part	events	normal	attacks	groups	time_min	time_max	attack_percent
0	train	3100	2942	158	1002	2026-01-01 00:16:27	2026-04-10 23:47:44	5.096774
1	validation	1034	981	53	678	2026-01-01 02:31:19	2026-04-10 23:46:44	5.125725
2	calibration	1034	981	53	671	2026-01-01 00:19:27	2026-04-10 23:43:44	5.125725
3	test	1137	1076	61	227	2026-04-11 01:45:36	2026-04-30 22:29:24	5.364996

В `train`, `validation` и `calibration` доли атак стали практически одинаковыми и соответствуют доле атак в исходной выборке.

Стратификация по `is_attack` сохраняет только общую долю атак, но не гарантирует одинаковое количество типов атак:

- `port_scan` ;
- `brute_force` ;
- `beaconing` ;
- `lateral_movement` ;
- `exfiltration` .

Доли атак различного типа в исходной выборке.

In [ ]:
```
source_props = df['scenario'].value_counts(normalize = True).reset_index()
source_props['proportion'] = source_props['proportion'].mul(100).round(3).astype
source_props
```

Out[ ]:

	scenario	proportion
0	normal	94.845%
1	brute_force	1.11%
2	port_scan	1.031%
3	lateral_movement	1.015%
4	beaconing	1.015%
5	exfiltration	0.983%

Доли атак каждого типа в различных выборках.

In [ ]:
```
# Функция для вычисления доли атак каждого типа
# в различных выборках
def get_attack_proportions_df(df, sample_name):
    props = df['scenario'].value_counts(normalize=True).reset_index()
    props.columns = ['scenario', 'proportion']
    props['proportion'] = props['proportion'].mul(100).round(3).astype(str) + '%
    props['sample'] = sample_name
    return props[['sample', 'scenario', 'proportion']]

train_props = get_attack_proportions_df(train, 'train')
calibration_props = get_attack_proportions_df(calibration, 'calibration')
val_props = get_attack_proportions_df(validation, 'validation')

distribution_summary = pd.concat([train_props, calibration_props, val_props], ig
print(source_props)
distribution_summary
```

```
          scenario proportion
0           normal    94.845%
1      brute_force      1.11%
2        port_scan     1.031%
3  lateral_movement     1.015%
4        beaconing      1.015%
5     exfiltration     0.983%
```

Out[ ]:

	sample	scenario	proportion
0	train	normal	95.018%
1	train	port_scan	1.161%
2	train	brute_force	1.161%
3	train	exfiltration	0.943%
4	train	lateral_movement	0.895%
5	train	beaconing	0.822%
6	calibration	normal	94.385%
7	calibration	beaconing	1.742%
8	calibration	lateral_movement	1.355%
9	calibration	exfiltration	1.162%
10	calibration	brute_force	0.774%
11	calibration	port_scan	0.581%
12	validation	normal	94.874%
13	validation	port_scan	1.257%
14	validation	exfiltration	1.161%
15	validation	lateral_movement	1.064%
16	validation	beaconing	0.87%
17	validation	brute_force	0.774%

Для сохранения пропорций атак различного типа исходной выборки нужно стратифицировать выборку по признаку `scenario`.

In [ ]:
```
remaining, calibration = train_test_split(
    source_df,
    test_size = 0.20,
    random_state = 42,
    shuffle = True,
    stratify = source_df["scenario"])

train, validation = train_test_split(
    remaining,
    test_size = 0.25,
    random_state = 42,
    shuffle = True,
    stratify = remaining["scenario"])
```

Доли атак каждого типа в различных выборках с учетом стратификации.

In [ ]:
```
train_props = get_attack_proportions_df(train, 'train')
calibration_props = get_attack_proportions_df(calibration, 'calibration')
val_props = get_attack_proportions_df(validation, 'validation')
```

```
distribution_summary = pd.concat([train_props, calibration_props, val_props], ig
print(source_props)
distribution_summary
```

```
          scenario proportion
0           normal    94.845%
1      brute_force      1.11%
2        port_scan     1.031%
3  lateral_movement     1.015%
4        beaconing      1.015%
5     exfiltration     0.983%
```

Out[ ]:

	sample	scenario	proportion
0	train	normal	94.903%
1	train	brute_force	1.065%
2	train	port_scan	1.032%
3	train	lateral_movement	1.0%
4	train	exfiltration	1.0%
5	train	beaconing	1.0%
6	calibration	normal	94.874%
7	calibration	beaconing	1.064%
8	calibration	port_scan	1.064%
9	calibration	brute_force	1.064%
10	calibration	exfiltration	0.967%
11	calibration	lateral_movement	0.967%
12	validation	normal	94.874%
13	validation	brute_force	1.161%
14	validation	port_scan	1.064%
15	validation	exfiltration	0.967%
16	validation	beaconing	0.967%
17	validation	lateral_movement	0.967%

Ограничения стратифицированного Hold-out.

- Сохраняет общую долю классов, но не гарантирует наличие каждого сценария атаки.
- Не учитывает группы связанных событий и временные зависимости.
- При очень редких классах стратификация может быть невозможна.
- Результат всё равно зависит от конкретного случайного разбиения.

Повторный Hold-out

Результат однократного Hold-out зависит от одного случайного разбиения. При другом значении параметра `random_state` может измениться:

- состав `train`, `calibration`, `validation`;
- выбранные гиперпараметры;
- итоговые выводы.

Повторный Hold-out выполняет случайное разбиение заданное количество раз и на каждой итерации независимо перемешивает данные и делит их на `train` и `validation`:

1. Случайно перемешиваются индексы объектов.
2. Первые n_{train} объектов используются для формирования `train`, остальные — для `validation`.
3. Модель обучается на `train`, метрика вычисляется на `validation`. Шаги 1–3 повторяются k раз с новым случайным перемешиванием.
4. Полученные метрики усредняются:

$$\overline{\text{Metric}}_{\text{validation}} = \frac{1}{k} \sum_{i=1}^{k} \text{Metric}_i.$$

После выбора модели и гиперпараметров проводится окончательная проверка на отдельно сохранённой `test`.

Для реализации повторного Hold-out можно использовать класс `ShuffleSplit` библиотеки `sklearn.model_selection`.

Основные параметры:

- `n_splits` (по умолчанию 10): сколько случайных разбиений сделать;
- `test_size`: доля или число объектов в `validation`;
- `train_size`: доля или число объектов в `train`;
- `random_state`: фиксирует случайность.

Создание разбиений.

In [ ]:
```
from sklearn.model_selection import ShuffleSplit

# Сначала резервируем calibration
# calibration не должна изменяться при каждом подборе гиперпараметров.
# Она понадобится после выбора конфигурации модели.
remaining, calibration = train_test_split(source_df,
                                          test_size = 0.20,
                                          shuffle = True,
                                          random_state = 42,
                                          stratify = source_df["is_attack"])
splitter = ShuffleSplit(n_splits = 5,
                        test_size = 0.25,
                        random_state = 42)
```

Чтобы посмотреть созданные разбиения используется метод `split()`.

Это итератор: на каждой итерации возвращает кортеж из индексов объектов(`train_indices` , `test_indices`).

Далее его используют в циклах при расчете метрик качества моделей: вычисляется метрика качества для каждого разбиения, после чего полученные значения усредняются.

In [ ]:
```
for split_number, (
    train_index,
    validation_index
) in enumerate(splitter.split(remaining), start = 1):
    train = remaining.iloc[train_index]
    validation = remaining.iloc[validation_index]

    print(f"Разбиение {split_number}: "
          f"train={len(train)}, "
          f"validation={len(validation)}")
```

```
Разбиение 1: train=3100, validation=1034
Разбиение 2: train=3100, validation=1034
Разбиение 3: train=3100, validation=1034
Разбиение 4: train=3100, validation=1034
Разбиение 5: train=3100, validation=1034
```

Ограничени метода ShuffleSplit.

- Одни события могут многократно попадать в `validation` , а другие — не попасть туда ни разу.
- `Validation` -наборы разных повторов пересекаются.
- Требует многократного обучения модели, поэтому увеличивает вычислительные затраты.
- Без групповых ограничений возможна утечка связанных событий.
- Не подходит для проверки устойчивости к изменениям во времени.

Основной недостаток метода `ShuffleSplit` : валидационные выборки разных повторений формируются независимо и поэтому могут пересекаться. Следовательно, одни события могут проверять модель несколько раз, а другие — ни разу. В задачах кибербезопасности это особенно важно, если среди не попавших в `validation` окажутся:

- редкие типы атак;
- необычные сетевые соединения;
- события отдельных пользователей или устройств;
- редкие комбинации признаков.

В таком случае итоговая оценка может недостаточно полно отражать поведение модели на всём датасете. Кроме того, результаты повторений нельзя считать полностью независимыми, поскольку их `validation` пересекаются.

Стратифицированный повторный Hold-out

Если необходимо сохранить доли классов исходной выборки используется класс `StratifiedShuffleSplit`.

In [ ]:
```
from sklearn.model_selection import StratifiedShuffleSplit
stratified_splitter = (StratifiedShuffleSplit(n_splits = 5,
                                              test_size = 0.25,
                                              random_state = 42))
```

Для разбиения передаются признаки и метки классов (для нашего примера `is_attack`).

In [ ]:
```
X_remaining = (remaining[MODEL_FEATURES].to_numpy())
y_remaining = (remaining["is_attack"].to_numpy())

stratified_shuffle_results = []

for split_number, (
    train_index,
    validation_index
) in enumerate(stratified_splitter.split(X_remaining, y_remaining),
    start=1):
    train = remaining.iloc[train_index]
    validation = remaining.iloc[validation_index]

    stratified_shuffle_results.append({"split": split_number,
                                       "train_events": len(train),
                                       "validation_events": len(validation),
                                       "train_attacks": int(train["is_attack"].s
                                       "validation_attacks": int(validation["is_
                                       "train_attack_percent": (train["is_attack
                                       "validation_attack_percent": (validation[
                                       "group_overlap": (count_group_overlap(tra

stratified_shuffle_results = (pd.DataFrame(stratified_shuffle_results))
stratified_shuffle_results
```

Out[ ]:

	split	train_events	validation_events	train_attacks	validation_attacks	train_attack_pe
0	1	3100	1034	158	53	5.09
1	2	3100	1034	158	53	5.09
2	3	3100	1034	158	53	5.09
3	4	3100	1034	158	53	5.09
4	5	3100	1034	158	53	5.09

Ограничения метода StratifiedShuffleSplit.

- Сохраняет доли классов, но не обязательно доли отдельных типов атак.
- Не гарантирует, что каждое событие хотя бы один раз окажется в validation.
- Не учитывает принадлежность событий одному инциденту, узлу или пользователю.
- Не сохраняет временную последовательность.

- Увеличивает вычислительные затраты из-за повторных запусков.

Bootstrap

При `bootstrap` из исходного набора данных размера n случайно выбирается n объектов с возвращением. Это означает, что после выбора объекта он остаётся доступным для повторного выбора.

Например:

Исходные индексы	Bootstrap
0	2
1	1
2	2
3	4
4	2

Объект 2 выбран три раза, а объекты 0 и 3 не выбраны.

Выбранные объекты 1, 2, 4 образуют выборку `train`, а не выбранные 0, 3 - `validation`, называемую `Out-of-Bag`.

В общем случае приблизительно:

- 63,2% уникальных объектов входят в `train`;
- 36,8% объектов остаются для `validation Out-of-Bag`.

При этом общее число строк в `train` равно n, поскольку многие объекты повторяются.

Таким образом, этот метод позволяет не создавать отдельно `validation` и `train`, поскольку в качестве вализационной выборки можно использовать `validation Out-of-Bag`.

Для примера используем данные после отделения `calibration`.

Библиотека `sklearn` предоставляет функцию `resample()`, поддерживающую выбор с возвращением и стратификацию.

In [ ]:
```
from sklearn.utils import resample
# Сохраняем исходные индексы строк.
remaining = remaining.reset_index(drop = True)

train = resample(remaining,
                 # Выбор с возвращением.
                 replace = True,
                 # Размер train равен размеру
                 # исходного набора.
                 n_samples = len(remaining),
```

```
                  random_state = 42)

# Индексы объектов, вошедших в train:
selected_indices = set(train.index)

# Формируем validation.
validation = remaining.loc[~remaining.index.isin(selected_indices)].copy()

display(train.head(2), validation.head(2))
```

	event_id	timestamp	window_start	window_end	host_id	host_role	user_id	ses
860	event-004479	2026-03-29 15:21:51	2026-03-29 15:16:51	2026-03-29 15:21:51	ws-023	workstation	user-043	r s
3772	event-002400	2026-02-18 07:04:41	2026-02-18 06:59:41	2026-02-18 07:04:41	ws-004	workstation	user-049	r s

	event_id	timestamp	window_start	window_end	host_id	host_role	user_id	session
0	event-004970	2026-04-06 05:29:30	2026-04-06 05:24:30	2026-04-06 05:29:30	ws-033	workstation	user-011	norm sessi 1
8	event-000141	2026-01-04 08:30:00	2026-01-04 08:25:00	2026-01-04 08:30:00	srv-07	server	user-017	norm sessi 0(

In [ ]:
```
print(f'Доля объектов в validation {round((len(validation)/len(remaining))*100,
```

```
Доля объектов в validation 37.98%
```

Для сохранения доли атак исходной выборки при использовании `resample` в параметр `stratify` передается необходимый признак: `stratify = remaining["is_attack"]`.

Ограничения метода Bootstrap.

- Одни объекты многократно дублируются в `train`, а другие не используются для обучения.
- Метод `Bootstrap` следует особенно осторожно применять к методам выявления аномалий, основанным на расстояниях и плотности. Так как в результате применения `Bootstrap` образуется много повторяющихся объектов, которые будут иметь иметь нулевые расстояния.
- При наличии связанных событий возможна утечка между `train` и `validation`.
- Обычный `bootstrap` предполагает независимость наблюдений, что часто нарушается в сетевом трафике.
- Не сохраняет временную структуру данных.

- Для корректной работы с инцидентами или временными рядами нужны групповой либо блочный `bootstrap`.

4.2. Кросс-валидация по блокам

При однократном Hold-out модель оценивается только на одном варианте разделения данных. Полученный результат может оказаться случайно слишком хорошим или слишком плохим.

Для получения устойчивого результата оценивания качества модели используется кросс-валидация по блокам.

Для демонстрации работы методов кросс-валиlации подготовим необходимые данные.

In [ ]:
```
from sklearn.model_selection import (KFold,
                                     StratifiedKFold,
                                     RepeatedKFold,
                                     RepeatedStratifiedKFold,
                                     LeaveOneOut,
                                     LeavePOut)

# Признаки, которые могут использоваться моделью обнаружения аномалий.
FEATURES = ["connections_per_min",
            "unique_destinations",
            "outbound_mb",
            "failed_logins",
            "duration_sec"]

# Полное отделение test
# В датасете final_test представляет события более позднего периода.
# Они не должны участвовать в кросс-валидации.
test = (df.loc[df["dataset_part"] == "final_test"].copy().reset_index(drop=True)

# События development первоначально помещаем в train.
train = (df.loc[df["dataset_part"] == "development"].copy().reset_index(drop=Tru

# Отделение calibration
# calibration отделяется до проведения кросс-валидации.
# Стратификация сохраняет приблизительную долю атак.
train, calibration = train_test_split(train,
                                      test_size = 0.20,
                                      stratify = train["is_attack"],
                                      random_state = 42)
train = train.reset_index(drop = True)
calibration = calibration.reset_index(drop = True)

# Матрица признаков и целевая переменная
X = train[FEATURES]
y = train["is_attack"]

print("Размеры выборок:")
print(f"train:       {len(train)}")
print(f"calibration: {len(calibration)}")
print(f"test:        {len(test)}")
```

```
print("\nКоличество атак:")
print(f"train:       {train['is_attack'].sum()}")
print(f"calibration: {calibration['is_attack'].sum()}")
print(f"test:        {test['is_attack'].sum()}")
```

```
Размеры выборок:
train:       4134
calibration: 1034
test:        1137

Количество атак:
train:       211
calibration: 53
test:        61
```

Вспомогательная функция анализа блоков, которая показывает:

- размер `train` и `validation`;
- количество атак;
- долю атак;
- число общих `validation_group`;
- число инцидентов, события которых оказались одновременно в `train` и `validation`.

In [ ]:
```
def analyze_splits(splitter, X, y, data, use_y = False):

    """
    Параметры
    ----------
    splitter: Объект KFold, StratifiedKFold или другого метода разбиения данных.
    X:        Матрица признаков (DataFrame или массив) — входные данные для моде
    y:        Вектор меток классов (Series или массив): 0 — нормальное событие,
    data:     Исходная таблица (DataFrame), содержащая не только признаки и метк
              (например, validation_group, incident_id), нужные для проверки «чи
    use_y:    Флаг, указывающий, нужно ли передавать метки классв (y) в метод sp
              Для стратифицированных методов (StratifiedKFold и т. п.) это обяза
              для обычных (KFold, GroupKFold) — не нужно (False).
    """
    rows = []
    if use_y:
        # Если флаг use_y равен True, значит, метод разбиения требует метки клас
        # Вызываем метод splitter.split() с передачей и признаков X, и меток y.
        # На выходе получаем итератор пар индексов (train_idx, validation_idx).
        splits = splitter.split(X, y)
    else:
        # Если use_y равен False, метки передавать не нужно (например, обычный K
        # Вызываем splitter.split() только с признаками X.
        splits = splitter.split(X)

        # Запускаем цикл по всем разбиениям, которые вернул splitter.
        # enumerate добавляет счётчик iteration (начиная с 1), а из splits получ
        # train_idx — индексы строк для обучающей выборки,
        # validation_idx — индексы строк для валидационной выборки.
    for iteration, (train_idx, validation_idx) in enumerate(splits, start=1):

        # По индексам train_idx выбираем соответствующие метки из y.
        # Получаем Series с метками для обучающей выборки.
```

```
        y_train = y.iloc[train_idx]

        # Аналогично — метки для валидационной выборки по индексам validation_id
        y_validation = y.iloc[validation_idx]

        # Группы связанных событий в train и validation.
        # Из исходной таблицы data по индексам train_idx берём колонку "validati
        # .dropna() убирает пропуски (NaN), чтобы не ломать логику.
        # set(...) превращает список значений в множество уникальных групп.
        train_groups = set(data.iloc[train_idx]["validation_group"].dropna()

        # То же самое для валидационной выборки: получаем множество уникальных v
        validation_groups = set(data.iloc[validation_idx]["validation_group"].dr

        # Идентификаторы реальных атакующих инцидентов.
        # Это нужно, чтобы проверить, не «просочились» ли одни и те же инциденты
        train_incidents = set(data.iloc[train_idx]["incident_id"].dropna())

        # Аналогично — уникальные incident_id для валидационной выборки.
        validation_incidents = set(data.iloc[validation_idx]["incident_id"].drop

        rows.append({
            # Номер текущей итерации (фолда) кросс-валидации.
            "iteration": iteration,
            # Размер обучающей выборки.
            "train_size": len(train_idx),
            # Размер валидационной выборки.
            "validation_size": len(validation_idx),
            # Количество атак в обучающей выборке: sum() по меткам 0/1.
            "train_attacks": int(y_train.sum()),
            # Количество атак в обучающей выборке: sum() по меткам 0/1.
            "validation_attacks": int(y_validation.sum()),
            # Доля атак в валидационной выборке: mean() для 0/1 даёт долю единиц
            "validation_attack_share": y_validation.mean(),
            # Число общих validation_group между train и validation:
            # Если значение > 0, значит, одни и те же группы попали и в train,
            # и в validation — это потенциальная утечка.
            "shared_validation_groups": len(train_groups & validation_groups),
            # Число общих incident_id между train и validation — аналогично,
            # проверка на утечки по инцидентам.
            "shared_incidents": len(train_incidents & validation_incidents)})

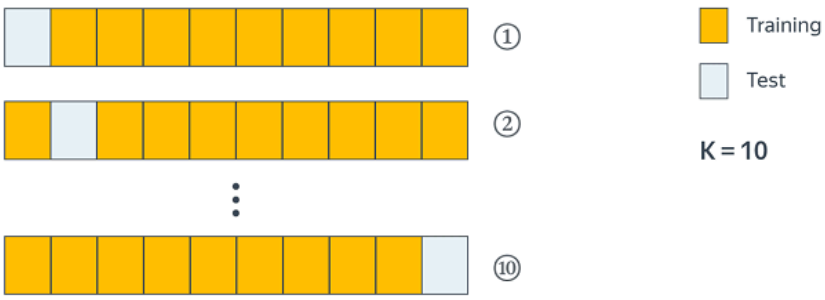
    return pd.DataFrame(rows)
```

K-Fold

K-fold представляет из себя следующий алгоритм:

1. Задается некоторое целое число k (обычно от 5 до 10), меньше числа наблюдений в датасете.
2. Датасет разбивается на k одинаковых частей. Эти части называются *фолдами*.
3. Далее происходит k итераций, во время каждой из которых один фолд выступает в роли валидационного множества, а объединение остальных — в роли тренировочного. Модель учится на $k - 1$ фолде и валидируется на оставшемся.

4. Рассчитывается итоговая метрика качества либо усреднением получившихся тестовых результатов, либо измеряется на отложенном тестовом множестве, не участвовавшем в кросс-валидации.



Кросс-валидация проводится только внутри `train`.

`test` нельзя включать в блоки, использовать для выбора гиперпараметров или просматривать после каждого эксперимента. Иначе он фактически превращается в ещё одну выборку `validation`.

Для реализации метода K-fold можно использовать класс `KFold` из `sklearn.model_selection`.

Основные параметры:

- `n_splits` (по умолчанию 5): число фолдов;
- `shuffle` (по умолчанию `False`): перемешивать ли данные перед разбиением. При `True` возможна утечка для временных данных. Для статических датасетов (сэмплы атак/норма без привязки ко времени) можно включать;
- `random_state`: фиксирует случайность при `shuffle=True`.

Для данных, расположенных в приблизительно хронологическом порядке, необходимо использовать: `shuffle = True`.

При `shuffle = False` блоки будут формироваться из соседних строк и могут значительно различаться по периодам наблюдения и сценариям атак.

Предположим, события в датасете записаны примерно в том порядке, в котором они произошли:

Номера строк	Период наблюдения	Преобладающий сценарий
1–1000	январь	нормальная работа сети
1001–2000	февраль	сканирование портов
2001–3000	март	подбор паролей
3001–4134	апрель	боковое перемещение и эксфильтрация

При использовании `shuffle = False` строки перед разделением не перемешиваются. Поэтому K-Fold формирует `validation` из последовательных участков таблицы:

Итерация	`validation`	Что может находиться в выборке
1	строки 1–1034	преимущественно январские события
2	строки 1035–2068	преимущественно февральские события
3	строки 2069–3101	преимущественно мартовские события
4	строки 3102–4134	преимущественно апрельские события

Получается, что блоки могут сильно отличаться друг от друга:

- по времени сбора данных;
- по нагрузке на сеть;
- по составу пользователей и узлов;
- по типам атак;
- по доле аномалий;
- по значениям сетевых признаков.

Например, в первом блоке может почти не быть атак, а в четвёртом могут находиться почти все события эксфильтрации. Тогда качество модели на разных итерациях будет сильно различаться не из-за случайности, а потому, что модель проверяется на разных периодах и разных сценариях.

Основным методом класса K- `fold` является `split()`, который генерирует индексы для разбиения данных на обучающую и валидационную выборки в рамках K-блочной кросс-валидации. Он не меняет сами данные — только возвращает списки (массивы) индексов строк, которые должны попасть в `train` и `validation` на каждой итерации. На выходе получаем итератор пар индексов (`train_idx`, `validation_idx`).

In [ ]:
```
kf = KFold(n_splits = 3, shuffle = True, random_state = 42)
for train_idx, test_idx in kf.split(X):
    X_tr = X.iloc[train_idx]
    X_val = X.iloc[test_idx]
    print(f'Количество объектов в train: {len(X_tr)}')
    print(X_tr[:3])
    print(f'Количество объектов в validation: {len(X_val)}')
    print(X_val[:3])
```

```
Количество объектов в train: 2756
   connections_per_min  unique_destinations  outbound_mb  failed_logins  \
1                51.97                12.81        8.823              0
2               117.42                43.15        0.824              0
3                61.84                17.11        4.567              0

   duration_sec
1          95.7
2          12.7
3          84.3
Количество объектов в validation: 1378
   connections_per_min  unique_destinations  outbound_mb  failed_logins  \
0                56.71                13.99        7.496              0
6                64.06                12.66        9.625              0
7                40.91                16.86        9.988              0

   duration_sec
0          65.5
6          69.6
7          51.2
Количество объектов в train: 2756
   connections_per_min  unique_destinations  outbound_mb  failed_logins  \
0                56.71                13.99        7.496              0
1                51.97                12.81        8.823              0
3                61.84                17.11        4.567              0

   duration_sec
0          65.5
1          95.7
3          84.3
Количество объектов в validation: 1378
    connections_per_min  unique_destinations  outbound_mb  failed_logins  \
2                117.42                43.15        0.824              0
13                84.73                 3.63        3.385              0
15                11.57                 1.60      176.779              0

    duration_sec
2           12.7
13          38.6
15         527.7
Количество объектов в train: 2756
   connections_per_min  unique_destinations  outbound_mb  failed_logins  \
0                56.71                13.99        7.496              0
2               117.42                43.15        0.824              0
6                64.06                12.66        9.625              0

   duration_sec
0          65.5
2          12.7
6          69.6
Количество объектов в validation: 1378
   connections_per_min  unique_destinations  outbound_mb  failed_logins  \
1                51.97                12.81        8.823              0
3                61.84                17.11        4.567              0
4                57.70                18.66        9.465              0

   duration_sec
1          95.7
3          84.3
4          92.9
```

Реализация.

In [ ]:
```
from sklearn.model_selection import KFold

kfold = KFold(n_splits = 5, shuffle = True, random_state = 42)
kfold_result = analyze_splits(splitter = kfold,
                              X = X,
                              y = y,
                              data = train,
                              use_y = False)
kfold_result
```

Out[ ]:

	iteration	train_size	validation_size	train_attacks	validation_attacks	validation_attacl
0	1	3307	827	171	40	0.
1	2	3307	827	163	48	0.
2	3	3307	827	164	47	0.
3	4	3307	827	173	38	0.
4	5	3308	826	173	38	0.

Размеры валидационных блоков практически одинаковы, но количество атак в них изменяется от 38 до 48.

K-Fold выравнивает количество событий, но не состав классов.

Проверка участия каждого события в валидационной выборке на каждой итерации.

In [ ]:
```
validation_count = np.zeros(len(train), dtype=int)

for _, validation_idx in kfold.split(X):
    # Для каждого фолда увеличиваем счётчик на 1 для всех объектов, попавших в v
    validation_count[validation_idx] += 1

# Выводим минимальное число раз, которое каждый объект побывал в validation.
# При K-Fold каждый объект должен попасть в validation ровно 1 раз,
# т.е. минимум должен быть равен 1.
print("Минимальное число попаданий в validation:",
      validation_count.min())

# Выводим максимальное число раз, которое каждый объект побывал в validation.
# Если max > 1, значит, есть объекты, которые чаще оказывались в validation — эт
# на ошибку логики или на использование метода с перекрытием (например, ShuffleS
print("Максимальное число попаданий в validation:",
      validation_count.max())
```

```
Минимальное число попаданий в validation: 1
Максимальное число попаданий в validation: 1
```

Главное преимущество K-Fold по сравнению с повторный Hold-out заключается в том, что модель обязательно проверяется на каждом событии датасета. В повторном Hold-out часть событий может проверяться несколько раз, а часть — ни разу.

Это особенно важно при обнаружении аномалий, потому что аномальных событий обычно мало. В повторном Hold-out некоторые редкие атаки могут случайно ни разу не попасть в `validation`.

Например, в датасете есть три события эксфильтрации. При повторном Hold-out одно из них может проверяться несколько раз, а два других — ни разу. Полученная оценка будет характеризовать работу модели только на одном варианте эксфильтрации. В K-Fold каждое из трёх событий обязательно будет проверено.

Ограничения K-Fold.

- Не сохраняет долю атак в каждом блоке.
- При малом числе атак некоторые блоки могут содержать слишком мало атак или не содержать их вообще.
- Не гарантирует присутствие каждого сценария атаки.
- Не учитывает связи между событиями одного инцидента, узла, пользователя или сетевой сессии.
- Не учитывает временной порядок.
- При большом количестве фолдов K увеличивается количество обучений модели.
- Для алгоритмов, чувствительных к локальной плотности, состав соседей может заметно меняться между блоками.

Stratified K-Fold

Stratified K-Fold является стратифицированным вариантом K-Fold. Он формирует фолды так, чтобы доля классов в каждом фолде была максимально близка к доле классов во всём `train`. Если в `train` доля атак составляет приблизительно 5,1%, то метод стремится сохранить около 5,1% атак в каждом `validation`.

Механизм работы:

- Все объекты группируются по меткам классов.
- Внутри каждого класса делается обычное K-Fold разбиение.
- Фолды собираются обратно: в каждый фолд попадает по одной части из каждого класса.
- В результате в каждом фолде пропорции классов почти совпадают с глобальными.

В библиотеке `sklearn` метод реализован классом `StratifiedKFold`.

In [ ]:
```
print(f'Доля атак в train: {np.round(train['is_attack'].mean()*100,2)}%')
```

```
Доля атак в train: 5.1%
```

In [ ]:
```
stratified_kfold = StratifiedKFold(n_splits = 5,
                                   shuffle = True,
                                   random_state = 42)
```

```
stratified_result = analyze_splits(splitter = stratified_kfold,
                                   X = X,
                                   y = y,
                                   data = train,
                                   use_y = True)

stratified_result[["iteration",
                   "train_size",
                   "train_attacks",
                   "validation_size",
                   "validation_attacks",
                   "validation_attack_share"]]
```

Out[ ]:

	iteration	train_size	train_attacks	validation_size	validation_attacks	validation_attacl
0	1	3307	169	827	42	0.
1	2	3307	169	827	42	0.
2	3	3307	169	827	42	0.
3	4	3307	168	827	43	0.
4	5	3308	169	826	42	0.

Количество атак стало практически одинаковым.

Стратификация по типам атак.

Обычный код: `stratified_kfold.split(X, y)` стратифицирует данные только по признаку `is_attack`. Он выравнивает классы «норма» и «атака», но не гарантирует исходные пропорции типов атак.

Распределение типов атак в `train`.

In [ ]:
```
print(f'Доля различных типов атак в train: {np.round(train['scenario'] \
                                                .value_counts(normalize=True)*100,2)
```

```
Доля различных типов атак в train: scenario
normal              94.90
brute_force          1.09
port_scan            1.06
exfiltration         1.06
lateral_movement     1.02
beaconing            0.87
Name: proportion, dtype: float64%
```

Если нужно учитывать сценарии, можно использовать столбец `scenario`.

In [ ]:
```
scenario_y = train["scenario"]

scenario_kfold = StratifiedKFold(n_splits = 3,
                                 shuffle = True,
                                 random_state = 42)

for iteration, (train_idx, validation_idx) in enumerate(
    scenario_kfold.split(X, scenario_y), start=1):
```

```
    print(f"\nИтерация {iteration}")
    print(f'{np.round(train.iloc[validation_idx]["scenario"].value_counts(normal
```

```
Итерация 1
scenario
normal              94.92
exfiltration         1.09
brute_force          1.09
lateral_movement     1.02
port_scan            1.02
beaconing            0.87
Name: proportion, dtype: float64%

Итерация 2
scenario
normal              94.92
port_scan            1.09
brute_force          1.09
lateral_movement     1.02
exfiltration         1.02
beaconing            0.87
Name: proportion, dtype: float64%

Итерация 3
scenario
normal              94.85
exfiltration         1.09
brute_force          1.09
port_scan            1.09
lateral_movement     1.02
beaconing            0.87
Name: proportion, dtype: float64%
```

Такая схема возможна только тогда, когда для каждого сценария имеется не менее K событий.

Ограничения Stratified K-Fold.

- Для стратификации необходимы метки классов.
- В полностью неразмеченном наборе данных метод непосредственно применить нельзя.
- Стратификация по `is_attack` не выравнивает отдельные типы атак.
- Число блоков не должно превышать число объектов самого редкого класса.
- Метод не учитывает связи между событиями одного инцидента.
- Не учитывает временной порядок.

Repeated K-Fold

Метод Repeated K-Fold повторяет процедуру K-Fold R раз. Перед каждым повторением выполняется новое случайное перемешивание данных.

Отличие от повторного Hold-out.

Внутри каждого повторения Repeated K-Fold:

- блоки `validation` не пересекаются;
- каждый объект попадает в `validation` ровно один раз.
- за R повторений каждый объект окажется в `validation` ровно R раз.

В библиотеке `sklearn` метод реализован классом `RepeatedKFold`.

Количество повторений - R задается параметром `n_repeats`.

In [ ]:
```
repeated_kfold = RepeatedKFold(n_splits = 5,
                               n_repeats = 3,
                               random_state = 42)

repeated_kfold_result = analyze_splits(splitter = repeated_kfold,
                                       X = X,
                                       y = y,
                                       data = train,
                                       use_y = False)

# Добавляем номер повторения и номер блока.
repeated_kfold_result["repeat"] = (
    (repeated_kfold_result["iteration"] - 1) // 5 + 1)

repeated_kfold_result["fold"] = (
    (repeated_kfold_result["iteration"] - 1) % 5 + 1)

repeated_kfold_result[["repeat",
                       "fold",
                       "validation_size",
                       "validation_attacks",
                       "validation_attack_share"]]
```

Out[ ]:

	repeat	fold	validation_size	validation_attacks	validation_attack_share
0	1	1	827	40	0.048368
1	1	2	827	48	0.058041
2	1	3	827	47	0.056832
3	1	4	827	38	0.045949
4	1	5	826	38	0.046005
5	2	1	827	44	0.053204
6	2	2	827	40	0.048368
7	2	3	827	41	0.049577
8	2	4	827	46	0.055623
9	2	5	826	40	0.048426
10	3	1	827	46	0.055623
11	3	2	827	34	0.041112
12	3	3	827	44	0.053204
13	3	4	827	46	0.055623
14	3	5	826	41	0.049637

Проверка числа попаданий в `validation`.

In [ ]:
```
validation_count = np.zeros(len(train), dtype=int)

for train_idx, validation_idx in repeated_kfold.split(X):
    validation_count[validation_idx] += 1

print(f'Минимальное число попаданий: {validation_count.min()}')
print(f'Максимальное число попаданий: {validation_count.max()}')
```

```
Минимальное число попаданий: 3
Максимальное число попаданий: 3
```

Ограничения Repeated K-Fold.

- Не сохраняет исходную долю атак в каждом блоке.
- Требует K·R обучений модели.
- Результаты разных повторений не являются полностью независимыми.
- Одни и те же события многократно используются в обучении и проверке.
- Не учитывает события одного инцидента, сессии или кампании.
- Не сохраняет временную последовательность.
- Увеличение числа повторений не исправляет систематическую утечку данных.

Repeated Stratified K-Fold

Метод Repeated Stratified K-Fold объединяет две идеи:

- сохраняет приблизительно одинаковую долю классов в каждом блоке;
- повторяет разбиение несколько раз с разной рандомизацией.

В библиотеке `sklearn` метод реализован классом `RepeatedStratifiedKFold`.

Для несбалансированных данных это обычно более устойчивый вариант случайной блочной кросс-валидации.

In [ ]:
```
repeated_stratified_kfold = RepeatedStratifiedKFold(
    n_splits = 5,
    n_repeats = 3,
    random_state = 42)

repeated_stratified_result = analyze_splits(
    splitter=repeated_stratified_kfold,
    X = X,
    y = y,
    data = train,
    use_y = True)

repeated_stratified_result["repeat"] = (
    (repeated_stratified_result["iteration"] - 1) // 5 + 1)

repeated_stratified_result["fold"] = (
    (repeated_stratified_result["iteration"] - 1) % 5 + 1)

repeated_stratified_result[["repeat",
                            "fold",
                            "validation_size",
                            "validation_attacks",
                            "validation_attack_share"]]
```

Out[ ]:

	repeat	fold	validation_size	validation_attacks	validation_attack_share
0	1	1	827	42	0.050786
1	1	2	827	42	0.050786
2	1	3	827	42	0.050786
3	1	4	827	43	0.051995
4	1	5	826	42	0.050847
5	2	1	827	42	0.050786
6	2	2	827	42	0.050786
7	2	3	827	42	0.050786
8	2	4	827	43	0.051995
9	2	5	826	42	0.050847
10	3	1	827	42	0.050786
11	3	2	827	42	0.050786
12	3	3	827	42	0.050786
13	3	4	827	43	0.051995
14	3	5	826	42	0.050847

Метод Repeated Stratified K-Fold имеет такие же ограничения, что и метод Stratified K-Fold.

Leave-One-Out (LOO)

Leave-One-Out (LOO), или Leave-One-Out Cross-Validation (LOOCV) — это стратегия кросс-валидации, где на каждой итерации:

- одно событие используется как `validation`;
- остальные $n - 1$ событий используются как `train`.

Обучение модели осуществляется n раз.

В библиотеке `sklearn` метод Leave-One-Out реализован классом `LeaveOneOut`.

Реализация.

С целью экономии памяти и времени применим функцию `islice()` из модуля `itertools`. Она позволяет брать «срез» (первые N элементов) из итератора, не превращая его в список. Это полезно, когда итератор очень большой. Например, для нашей задачи запуск обучения модели 4134 раза может занять много времени.

In [ ]:
```
from itertools import islice

loo = LeaveOneOut()
```

```
# метод get_n_splits(X) возвращает количество разбиений (равно числу строк в X),
# не выполняя само разбиение — это быстрый способ узнать размер задачи.
print("Количество итераций:", loo.get_n_splits(X))

# Показываем только первые три итерации.
for iteration, (train_idx, validation_idx) in enumerate(
    islice(loo.split(X), 3), start=1):
    # Запускаем цикл только по первым 3 итерациям из всего итератора loo.split(X
    # islice(loo.split(X), 3) берёт ровно 3 первых разбиения и затем прекращает

    print(f"Итерация {iteration}: "
          f"train = {len(train_idx)}, "
          f"validation = {len(validation_idx)}, "
          f"event_id = "
          f"{train.iloc[validation_idx[0]]['event_id']}")
```

```
Количество итераций: 4134
Итерация 1: train = 4133, validation = 1, event_id = event-004970
Итерация 2: train = 4133, validation = 1, event_id = event-000945
Итерация 3: train = 4133, validation = 1, event_id = event-001126
```

Как оценивать качество модели при использовании LOO?

Вычислять метрики качества отдельно на каждой итерации нельзя, так как `validation` содержит только одно событие.

Сначала нужно собрать прогнозы для всех событий, после чего вычислять метрики.

In [ ]:
```
out_of_fold_prediction = np.zeros(len(train), dtype = int)

for train_idx, validation_idx in loo.split(X):

    # Здесь должна выполняться следующая последовательность:
    #
    # 1. Масштабирование только на train_idx.
    # 2. Обучение модели только на train_idx.
    # 3. Настройка порога по calibration.
    # 4. Прогноз для единственного события validation_idx.
    #
    # out_of_fold_prediction[validation_idx] = prediction
    pass

# После заполнения out_of_fold_prediction метрики вычисляются
# один раз по всем вневыборочным прогнозам:
#
# metric(y, out_of_fold_prediction)
```

Ограничения Leave-One-Out.

- Требуется n обучений модели.
- Вычислительно непрактичен для больших журналов событий.
- `validation` из одного события не позволяет вычислять большинство метрик на отдельной итерации.
- Результаты соседних моделей очень похожи, поскольку их `train` различаются только одним событием.
- Оценка может иметь высокую дисперсию.

- Метод чувствителен к дубликатам и связанным событиям.
- Не учитывает инциденты, узлы, пользователей и время.
- Обычно не даёт практического преимущества перед 5- или 10-блочной кросс-валидацией.

Leave-P-Out (LPO)

В Leave-P-Out (LPO) на каждой итерации в `validation` помещаются все возможные комбинации из p событий. Остальные $n - p$ событий используются как `train`.

Пусть у нас есть `train` из n объектов. Leave-P-Out делает следующее:

- Рассматривает каждую возможную подвыборку размера p как `validation`.
- Для каждой такой подвыборки обучает модель на оставшихся $n - p$ объектах.
- Оценивает качество модели для каждой комбинации подвыборок.
- Итоговая оценка вычисляется усреднением по всем таким комбинациям.

Число вариантов разбиения (количество обучений n_{fiy}):

$$n_{fit} = \frac{n!}{p! \cdot (n-p)!}\,.$$

Пример.

Пусть у нас есть 4 объекта с индексами: [0, 1, 2, 3]. Применим для разделения LPO с $p = 2$, то есть на каждой итерации будем брать в `validation` ровно 2 объекта, а на `train` оставлять остальные 2.

Число комбинаций из 4 по 2:

$$n_{fit} = \frac{4!}{2! \cdot (4-2)!} = 6\,.$$

Итерация	validation (p=2)	train (n−p=2)	Что происходит
1	`[0, 1]`	`[2, 3]`	Обучаем на 2 и 3, тестируем на 0 и 1
2	`[0, 2]`	`[1, 3]`	Обучаем на 1 и 3, тестируем на 0 и 2
3	`[0, 3]`	`[1, 2]`	Обучаем на 1 и 2, тестируем на 0 и 3
4	`[1, 2]`	`[0, 3]`	Обучаем на 0 и 3, тестируем на 1 и 2
5	`[1, 3]`	`[0, 2]`	Обучаем на 0 и 2, тестируем на 1 и 3
6	`[2, 3]`	`[0, 1]`	Обучаем на 0 и 1, тестируем на 2 и 3

Для нашего набора данных при $n = 4134$ и $p = 2$ получем количество итераций (обучений модели):

$$n_{fit} = \frac{4134!}{2! \cdot (4134-2)!} = 8542911\,.$$

При $p = 3$:

$$n_{fit} = 11766436084\,.$$

Поэтому полный запуск Leave-P-Out на этом датасете практически невозможен.

В библиотеке `sklearn` метод Leave-P-Out реализован классом `LeavePOut`.

Реализация.

In [ ]:
```
leave_two_out = LeavePOut(p=2)

# Получаем только первое разбиение.
# Все 8,5 миллиона вариантов перебирать не будем.
train_idx, validation_idx = next(leave_two_out.split(X))

print("Размер train:", len(train_idx))
print("Размер validation:", len(validation_idx))
print("Количество вариантов при p=2:", leave_two_out.get_n_splits(X))

print("\nСобытия validation:")
print(train.iloc[validation_idx][["event_id", "scenario", "is_attack"]])
```

```
Размер train: 4132
Размер validation: 2
Количество вариантов при p=2: 8542911

События validation:
       event_id scenario  is_attack
0  event-004970   normal          0
1  event-000945   normal          0
```

Ограничения Leave-P-Out.

- Количество обучений растёт комбинаторно.
- Практически неприменим даже для наборов среднего размера.
- Выборки `validation` сильно пересекаются.
- Полученные оценки не являются независимыми.
- При малом p в большинстве `validation` могут отсутствовать атаки.
- При большом p вычислительная сложность становится ещё выше.
- Не учитывает группировку событий по инцидентам.
- Не учитывает временной порядок.
- Обычно применяется только для очень маленьких наборов данных или теоретического анализа.

Общие правила корректной реализации подходов повторного случайного разбиения и кросс-валидации.

На каждой итерации все обучаемые преобразования должны настраиваться заново только по текущему `train_i`:

- заполнение пропусков;

- масштабирование;
- кодирование категорий;
- отбор признаков;
- обучение модели.

Неправильно заранее делать преобразования для всего набор `train`. В этом случае средние значения, стандартные отклонения и другие параметры вычисляются с использованием будущих `validation`, что приводит к утечке данных.

В реальных системах кибербезопасности этих методов часто недостаточно. Они предполагают относительную независимость событий и не предотвращают попадание частей одного инцидента одновременно в `train` и `validation`. Поэтому при наличии `validation_group`, `incident_id`, `campaign_id`, `host_id` или временной зависимости необходимо переходить к групповой либо временной кросс-валидации.

4.3. Групповые способы разбиения

Обычные методы `KFold`, `StratifiedKFold`, `ShuffleSplit`, `LeavePOut` рассматривают каждую строку датасета как отдельное независимое событие.

В данных кибербезопасности это предположение часто нарушается. Несколько строк могут относиться:

- к одной сетевой сессии;
- к одному пользователю;
- к одному компьютеру;
- к одному инциденту;
- к одной атакующей кампании;
- к одному вредоносному файлу;
- к одному источнику журналов.

Например, сканирование портов может быть представлено шестью последовательными событиями:

Событие	Инцидент	Число соединений
event_1	incident_17	170
event_2	incident_17	184
event_3	incident_17	191
event_4	incident_17	203
event_5	incident_17	198
event_6	incident_17	215

При обычном случайном разбиении, например, четыре события могут попасть в `train`, а два — в `validation`.

Модель фактически уже видела часть того же инцидента. События в `validation` будут очень похожи на события из `train`, поэтому оценка качества окажется завышенной.

Групповое разбиение вводит правило: все события одной группы должны целиком попасть либо в `train`, либо в `validation`.

Например, если группой является `incident_17`, все шесть событий этого инцидента перемещаются как единое целое.

В нашем датасете имеются несколько столбцов, которые можно использовать как идентификаторы групп.

Столбец	Что считается группой	Какой сценарий проверки моделируется
`validation_group`	Сессия для нормы или инцидент для атаки	Новые сессии и новые инциденты
`session_id`	Одна сетевая сессия	Работа на ранее не встречавшихся сессиях
`incident_id`	Один атакующий инцидент	Работа на новых инцидентах
`campaign_id`	Одна атакующая кампания	Работа на атаках новой кампании
`host_id`	Один компьютер	Работа на ранее не встречавшихся узлах
`user_id`	Один пользователь	Работа с ранее не встречавшимися пользователями

В качестве основного группирующего поля выступает `validation_group`.

Это обусловлено тем, что у нормальных событий обычно нет `incident_id`. Если непосредственно использовать его как группу, у всех нормальных событий будет пропущенное значение.

Если заменить все пропуски на `normal`, то все нормальные события превратятся в одну большую группу. Их невозможно будет нормально распределить между блоками.

Поэтому на практике, так же как и в учебном датасете, рекомендуется заранее создавать поля типа `validation_group`:

$$validation_group_i = \begin{cases} incident_id_i, & \text{если событие является атакой;} \\ session_id_i, & \text{если событие является нормальным.} \end{cases}$$

Таким образом:

- все события одного атакующего инцидента остаются вместе;
- все события одной нормальной сессии остаются вместе;

- каждая строка имеет идентификатор группы.

При использовании групповых методов нельзя сначала случайно разделить строки на `train` и `calibration`, а затем применять групповой метод только внутри `train`.

В этом случае части одной группы уже могут оказаться в разных выборках. Поэтому `calibration` также следует отделять с учётом групп.

In [ ]:
```
FEATURES = ["connections_per_min",
            "unique_destinations",
            "outbound_mb",
            "duration_sec"]

# Отделяем заранее выделенную тестовую выборку.
test = (df.loc[df["dataset_part"] == "final_test"].copy().reset_index(drop=True)

# Выбираем данные для обучения, валидации и калибровки.
development = (df.loc[df["dataset_part"] == "development"].copy().reset_index(dr

# Проверяем, что у каждого события указан идентификатор группы.
assert development["validation_group"].notna().all()

# Получаем уникальные идентификаторы групп.
# Каждая группа представлена в этом списке один раз.
group_ids = development["validation_group"].drop_duplicates()

# Случайно выбираем 20% групп для калибровки.
# random_state обеспечивает воспроизводимость выбора.
calibration_groups = group_ids.sample(frac = 0.20, random_state = 42)

# Определяем, какие события относятся к выбранным группам.
calibration_mask = development["validation_group"].isin(calibration_groups)

# Все события выбранных групп помещаем в calibration.
calibration = (development.loc[calibration_mask].copy().reset_index(drop=True))

# Остальные события оставляем для обучения и валидации.
# Далее групповые методы будут разделять эту часть
# на train и validation в каждой итерации.
train = (development.loc[~calibration_mask].copy().reset_index(drop=True))

# Подготавливаем признаки, метки классов и идентификаторы групп.
X = train[FEATURES]
y = train["is_attack"]
groups = train["validation_group"]

print("Размер train:", len(train))
print("Размер calibration:", len(calibration))
print("Количество групп в train:", groups.nunique())

# Проверяем отсутствие общих групп.
shared_groups = (set(train["validation_group"]) & set(calibration["validation_gr

print("Общих групп между train и calibration:", len(shared_groups))
```

```
Размер train: 4089
Размер calibration: 1079
Количество групп в train: 818
Общих групп между train и calibration: 0
```

Вспомогательная функция анализа разбиений, которая для каждого разбиения показывает:

- размер `train` и `validation`;
- количество групп;
- количество атак;
- долю атак;
- пересечение групп;
- пересечение инцидентов;
- пересечение кампаний.

In [ ]:
```python
def analyze_group_splits(splitter, X, y, groups, data):

    rows = []

    for iteration, (train_idx, validation_idx) in enumerate(
        splitter.split(X, y, groups), start=1):
        # Получаем метки для обучающей и валидационной выборок.
        y_train = y.iloc[train_idx]
        y_validation = y.iloc[validation_idx]

        # Извлекаем уникальные группы в train и validation.
        train_groups = set(groups.iloc[train_idx])
        validation_groups = set(groups.iloc[validation_idx])

        # Для кибербезопасности важно проверить, не «перетекают» ли между train/
        # не только группы, но и более высокоуровневые сущности: инциденты и ком
        # Берем столбцы incident_id и campaign_id из исходного DataFrame data,
        # отбираем строки по индексам train/validation, убираем пропуски через d
        # затем превращаем в set, чтобы получить уникальные идентификаторы.
        train_incidents = set(data.iloc[train_idx]["incident_id"].dropna())
        validation_incidents = set(data.iloc[validation_idx]["incident_id"].drop

        train_campaigns = set(data.iloc[train_idx]["campaign_id"].dropna())
        validation_campaigns = set(data.iloc[validation_idx]["campaign_id"].drop

        rows.append({"iteration": iteration,
                     "train_size": len(train_idx),
                     "train_groups": len(train_groups),
                     "train_attacks": int(y_train.sum()),
                     "validation_size": len(validation_idx),
                     "validation_groups": len(validation_groups),
                     "validation_attacks": int(y_validation.sum()),
                     "validation_attack_share": y_validation.mean(),
                     "shared_groups": len(train_groups & validation_groups),
                     "shared_incidents": len(train_incidents & validation_incide
                     "shared_campaigns": len(train_campaigns & validation_campai

    return pd.DataFrame(rows)
```

In [ ]:
```python
from sklearn.model_selection import (GroupKFold,
                                     StratifiedGroupKFold,
```

```
                                    GroupShuffleSplit,
                                    LeaveOneGroupOut,
                                    LeavePGroupsOut)
```

GroupKFold

GroupKFold является групповым аналогом обычного K-Fold.

Алгоритм:

1. Для каждого события задаётся идентификатор группы.
2. Группы разделяются на K наборов.
3. На каждой итерации один набор групп используется как `validation`.
4. Остальные группы используются как `train`.
5. Каждая группа становится валидационной ровно один раз.

Предположим, у нас есть 12 сетевых событий, объединённых в 6 групп. Каждая группа — одна сессия из двух связанных событий.

Группа	События
G1	1, 2
G2	3, 4
G3	5, 6
G4	7, 8
G5	9, 10
G6	11, 12

Применим GroupKFold с $k = 3$.

Сначала группы распределяются по трём блокам. Например:

Блок 1	Блок 2	Блок 3
G1, G2	G3, G4	G5, G6

Затем выполняются три итерации:

Итерация	Группы в `train`	Группы в `validation`
1	G3, G4, G5, G6	**G1, G2**
2	G1, G2, G5, G6	**G3, G4**
3	G1, G2, G3, G4	**G5, G6**

На каждой итерации модель обучается заново на восьми событиях и проверяется на четырёх.

В библиотеке `sklearn` GroupKFold реализован классом `GroupKFold`.

Если параметр `shuffle = True`, то перед распределением по блокам группы перемешиваются. При этом все события каждой группы остаются вместе.

В течение заданного количества итераций (k) эти блоки уже не меняются: сначала валидационным становится первый, затем второй, затем третий. Нового перемешивания перед каждой итерацией нет.

In [ ]:
```
group_kfold = GroupKFold(n_splits = 5,
                         shuffle = True,
                         random_state = 42)

group_kfold_result = analyze_group_splits(splitter = group_kfold,
                                          X=X, y=y,
                                          groups = groups,
                                          data = train)
group_kfold_result[["iteration",
                    "train_size",
                    "validation_size",
                    "validation_groups",
                    "validation_attacks",
                    "validation_attack_share",
                    "shared_groups",
                    "shared_incidents"]]
```

Out[ ]:

	iteration	train_size	validation_size	validation_groups	validation_attacks	validation_
0	1	3247	842	164	30	
1	2	3296	793	164	29	
2	3	3290	799	164	48	
3	4	3239	850	163	16	
4	5	3284	805	163	42	

Таким образом, ни одна группа и ни один атакующий инцидент не разделены между `train` и `validation` (`shared_groups = 0` и `shared_incidents = 0`).

При этом количество атак изменяется от 27 до 53. `GroupKFold` обеспечивает разделение групп, но не исходный баланс классов.

Проверка покрытия групп.

In [ ]:
```
validation_group_count = {group: 0 for group in groups.unique()}

for train_idx, validation_idx in group_kfold.split(X, y, groups):
    for group in groups.iloc[validation_idx].unique():
        validation_group_count[group] += 1

print("Минимальное число попаданий группы в validation:",
      min(validation_group_count.values()))

print("Максимальное число попаданий группы в validation:",
      max(validation_group_count.values()))
```

```
Минимальное число попаданий группы в validation: 1
Максимальное число попаданий группы в validation: 1
```

Как и в обычном K-Fold, каждая группа обязательно попадает в валидационную выборку ровно один раз.

Ограничения GroupKFold.

- Не сохраняет одинаковую долю атак в блоках.
- Некоторые сценарии атак могут отсутствовать в отдельных `validation`.
- Размеры блоков могут различаться из-за неодинакового размера групп.
- Результат зависит от выбранного определения группы.
- Нельзя создать больше блоков, чем имеется групп.
- Одна очень большая группа может сделать блоки сильно несбалансированными.
- Не учитывает временной порядок.
- Разделение по инцидентам не исключает пересечение атакующих компаний.
- При малом количестве атакующих групп оценка качества может быть нестабильной.

StratifiedGroupKFold

StratifiedGroupKFold - это кросс-валидатор, который одновременно:

- не разрывает группы: все объекты одной группы целиком попадают либо в `train`, либо в `validation` (как в GroupKFold);
- сохраняет пропорции классов: в каждом фолде доля положительных/отрицательных примеров примерно такая же, как во всём датасете (как в StratifiedKFold).

Это критично, когда в данных есть и группировка (например, по хостам/сессиям), и дисбаланс классов (атаки vs норма) и нужно избежать утечки по группам и при этом не получить фолд, где вообще нет атак.

Предположим есть 12 объектов разбитых на 4 группы.

idx	group	y (класс)
0	1	0
1	1	0
2	1	1
3	2	0
4	2	1
5	2	1
6	3	1
7	3	0

idx	group	y (класс)
8	3	0
9	4	0
10	4	0
11	4	1

Доля атак в этой выборке составляет примерно 41.7%.

Применим для разделения этих данных StratifiedGroupKFold с $k = 3$ (3 фолда).

Наша цель: в каждом из 3 фолдов получить примерно такую же долю атак, при этом не разрывая группы.

Алгоритм "пытается" подобрать такие наборы групп в `validation`, чтобы доля класса 1 в `validation` была близка к 41.7% и при этом все группы были целыми.

Один из возможных корректных вариантов разбиения:

Фолд	validation группы	validation объекты (idx)	validation: класс 1 / всего	доля атак в validation	train группы	train объекты (idx)	trai кла 1 / все
1	[1, 4]	[0,1,2,9,10,11]	2 / 6	33.3%	[2, 3]	[3,4,5,6,7,8]	3 /
2	[2]	[3,4,5]	2 / 3	66.7%	[1,3,4]	[0,1,2,6,7,8,9,10,11]	3 /
3	[3]	[6,7,8]	1 / 3	33.3%	[1,2,4]	[0,1,2,3,4,5,9,10,11]	4 /

В разных фолдах доля атак в `validation` колеблется от 33% до 67%, но это лучшее, что можно сделать при заданных группах.

Из этого примера видно, что слово «пытается» здесь важно. Если одна большая группа (группа 2 в примере) содержит много атак по сравнению с другими гркппами, точно выровнять доли классов, не разделив группу, невозможно.

В библиотеке `sklearn` StratifiedGroupKFold реализован классом `StratifiedGroupKFold`.

In [ ]:
```
stratified_group_kfold = StratifiedGroupKFold(n_splits = 5,
                                              shuffle = True,
                                              random_state = 42)

stratified_group_result = analyze_group_splits(splitter = stratified_group_kfold
                                               X = X, y = y,
                                               groups = groups,
                                               data = train)
stratified_group_result[["iteration",
                         "train_size",
                         "validation_size",
                         "validation_groups",
```

```
                                "validation_attacks",
                                "validation_attack_share",
                                "shared_groups",
                                "shared_incidents"]]
```

Out[ ]:

	iteration	train_size	validation_size	validation_groups	validation_attacks	validation_
0	1	3270	819	163	31	
1	2	3255	834	163	28	
2	3	3251	838	165	35	
3	4	3313	776	163	24	
4	5	3267	822	164	47	

Ограничения StratifiedGroupKFold.

- Для стратификации нужны метки классов.
- В полностью неразмеченном наборе метод применить нельзя.
- Точное сохранение долей невозможно при крупных или неоднородных группах.
- Баланс нормы и атак не гарантирует баланс отдельных сценариев атак.
- Редкий сценарий может отсутствовать в некоторых блоках.
- Метод не учитывает временной порядок.
- Стратификация может быть нестабильной при небольшом числе атакующих групп.

GroupShuffleSplit

Сплитер GroupShuffleSplit является групповым аналогом повторного Hold-out.

На каждой итерации метод случайно выбирает часть групп для `validation`. Оставшиеся группы используются как `train`.

Главное отличие от обычного ShuffleSplit: параметр `test_size` задаёт долю или количество групп, а не отдельных событий.

Например: `test_size=0.20` означает, что в `validation` будет помещено 20% групп. Доля строк может отличаться от 20%, если группы имеют разные размеры.

В библиотеке `sklearn` GroupShuffleSplit реализован классом `GroupShuffleSplit`.

In [ ]:

```
group_shuffle = GroupShuffleSplit(n_splits = 5,
                                  test_size = 0.20,
                                  random_state = 42)

group_shuffle_result = analyze_group_splits(splitter = group_shuffle,
                                            X = X, y = y,
                                            groups = groups,
                                            data = train)
```

```
group_shuffle_result[["iteration",
                      "train_size",
                      "validation_size",
                      "validation_groups",
                      "validation_attacks",
                      "validation_attack_share",
                      "shared_groups",
                      "shared_incidents"]]
```

Out[ ]:

	iteration	train_size	validation_size	validation_groups	validation_attacks	validation_
0	1	3247	842	164	30	
1	2	3262	827	164	22	
2	3	3276	813	164	32	
3	4	3262	827	164	30	
4	5	3247	842	164	7	

Внутри каждого разбиения группы не пересекаются. Но наборы `validation_groups` разных итераций могут пересекаться.

Проверка пересечения `validation_groups`.

In [ ]:

```
validation_groups_list = []
validation_rows = set()

for train_idx, validation_idx in group_shuffle.split(
    X, y, groups
):
    validation_groups_list.append(
        set(groups.iloc[validation_idx])
    )

    validation_rows.update(validation_idx)

for i in range(len(validation_groups_list) - 1):
    overlap = (validation_groups_list[i] & validation_groups_list[i + 1])

    print(
        f"Общие группы validation "
        f"между итерациями {i + 1} и {i + 2}: "
        f"{len(overlap)}"
    )

all_validation_groups = set().union(*validation_groups_list)

print("Хотя бы один раз попали в validation:",
      len(all_validation_groups), "групп из", groups.nunique())

print("Хотя бы один раз попали в validation:",
      len(validation_rows), "событий из", len(train))
```

```
Общие группы validation между итерациями 1 и 2: 35
Общие группы validation между итерациями 2 и 3: 29
Общие группы validation между итерациями 3 и 4: 33
Общие группы validation между итерациями 4 и 5: 32
Хотя бы один раз попали в validation: 555 групп из 818
Хотя бы один раз попали в validation: 2812 событий из 4089
```

Следовательно, после пяти повторений:

- некоторые группы использовались в `validation` несколько раз;
- 263 группы ни разу не использовались в `validation`;
- около 1370 событий ни разу не проверялись.

Это нормальное свойство повторного группового Hold-out.

Ограничения GroupShuffleSplit.

- Не гарантирует, что каждая группа попадёт в `validation`.
- validation разных повторений могут пересекаться.
- Одни группы проверяются несколько раз, другие — ни разу.
- Не сохраняет долю атак.
- При малом количестве атакующих групп возможны блоки почти без атак.
- Не учитывает временную последовательность.

Leave-One-Group-Out

В Leave-One-Group-Out на каждой итерации одна группа используется как `validation`, а все остальные группы — как `train`.

В отличие от обычного Leave-One-Out, исключается не одна строка, а целая группа связанных событий.

В библиотеке `sklearn` метод реализован классом `LeaveOneGroupOut`.

Можно использовать различные варианты применения этого метода.

Вариант 1. Исключение одной `validation_group`.

In [ ]:
```python
logo = LeaveOneGroupOut()

# Метод get_n_splits позволяет получить количество итераций (разбиений).
# Метод не выполняет само разбиение, а только считает, сколько уникальных групп
# Аргументы X и y здесь используются формально, так как для для подсчёта
# реально используется только groups: число разбиений равно числу уникальных гру
number_of_splits = logo.get_n_splits(X, y, groups = groups)

print("Количество разбиений:", number_of_splits)
```

```
Количество разбиений: 818
```

Каждый `validation` будет содержать одну нормальную сессию или один атакующий инцидент.

Большинство отдельных `validation` содержит только один класс. Поэтому метрики качества модели нельзя содержательно рассчитывать отдельно для каждой итерации. Сначала необходимо собрать все вневыборочные прогнозы, а затем один раз вычислить метрики по объединённым результатам.

Вариант 2. Исключение одного компьютера.

Более содержательный вариант — использовать `host_id`.

In [ ]:
```
host_groups = train["host_id"]
logo_host = LeaveOneGroupOut()

print("Количество компьютеров:", host_groups.nunique())
print("Количество разбиений:", logo_host.get_n_splits(X, y,
                                                      groups = host_groups))
```

```
Количество компьютеров: 48
Количество разбиений: 48
```

На каждой итерации модель проверяется на компьютере, который полностью отсутствовал в `train`.

In [ ]:
```
# Запускаем цикл по разбиениям кросс-валидации,
# но ограничиваемся только первыми 5 итерациями.
# train_idx — индексы строк, попадающих в обучающую выборку,
# validation_idx — индексы строк, попадающих в валидационную
# выборку (все строки одной группы-хоста).
# islice(..., 5) берёт из итератора только первые 5 разбиений.
for iteration, (train_idx, validation_idx) in enumerate(
    islice(logo_host.split(X, y, host_groups), 5), start = 1):

    # Извлекаем идентификатор хоста (группу) для текущей валидационной выборки.
    # host_groups.iloc[validation_idx] — выбираем значения host_groups по индекс
    # .iloc[0] — берём первое значение (в LeaveOneGroupOut все строки validation
    # к одному хосту, поэтому любое значение из этого среза даст нужный host_id)
    # В переменной host теперь хранится идентификатор хоста,
    # который на этой итерации выступает в роли «нового» (тестируемого).
    host = host_groups.iloc[validation_idx].iloc[0]

    print(
        f"Итерация {iteration}: "
        f"host_id = {host}, "
        f"validation = {len(validation_idx)}, "
        f"атак = {int(y.iloc[validation_idx].sum())}")
```

```
Итерация 1: host_id = srv-01, validation = 47, атак = 0
Итерация 2: host_id = srv-02, validation = 98, атак = 0
Итерация 3: host_id = srv-03, validation = 118, атак = 0
Итерация 4: host_id = srv-04, validation = 77, атак = 0
Итерация 5: host_id = srv-05, validation = 57, атак = 0
```

Такая схема отвечает на вопрос: сможет ли модель обнаруживать аномалии на ранее не встречавшемся компьютере?

Ограничения Leave-One-Group-Out.

- Требуется столько обучений, сколько имеется групп.

- При большом числе групп вычислительные затраты высоки.
- Размеры validation могут сильно различаться.
- Некоторые группы могут содержать только нормальные события или только атаки.
- Метрики отдельных итераций могут быть неопределены или нестабильны.
- Результат сильно зависит от выбранной группировки.
- Не учитывает направление времени.
- Одна необычная группа может сильно повлиять на итоговый результат.

4.4. Временные способы разбиения

События кибербезопасности почти всегда имеют временной порядок:

- сетевые соединения поступают последовательно;
- поведение пользователей изменяется в течение дня;
- конфигурация инфраструктуры меняется;
- появляются новые приложения и узлы;
- злоумышленники изменяют тактику;
- меняется интенсивность сетевого трафика;
- возникают новые типы атак.

Предположим, модель обучается на событиях июля и будет использоваться в августе. Проверять её нужно на событиях августа, то есть на данных, возникших после обучающих.

При случайном разбиении возможна противоположная ситуация:

- события августа попадают в `train`;
- события июля попадают в `validation`.

Получается, что модель обучается на «будущем» и проверяется на «прошлом». В реальной эксплуатации это невозможно.

Главное правило временного разбиения:

$$max(t_{\text{train}}) < min(t_{\text{validation}}),$$

где:

- t_{train} — моменты времени событий в `train`;
- $t_{\text{validation}}$ — моменты времени событий в `validation`.

То есть все события `train` должны произойти раньше всех событий `validation`.

В задачах кибербезопасности чаще всего применяются следующие временные способы разбиения:

- хронологический Hold-out;
- расширяющееся окно — expanding window, или walk-forward validation;
- скользящее окно — sliding/rolling window.

Методы с временным зазором, purge и embargo целесообразно рассматривать отдельно, поскольку они решают дополнительную задачу устранения пересечения событий на границах интервалов.

Перед любым временным разбиением события необходимо отсортировать по возрастанию времени.

In [ ]:
```
from sklearn.model_selection import TimeSeriesSplit

FEATURES = ["connections_per_min",
            "unique_destinations",
            "outbound_mb",
            "failed_logins",
            "duration_sec"]

# Полное отделение test
# test уже содержит наиболее поздний период.
test = (df.loc[df["dataset_part"] == "final_test"]
      .sort_values("timestamp")
      .reset_index(drop=True))

# Остальные события сначала сортируются по времени.
data = (df.loc[df["dataset_part"] == "development"]
      .sort_values("timestamp")
      .reset_index(drop=True))

print("Период исходных данных:")
print(data["timestamp"].min())
print(data["timestamp"].max())

print("\nПериод test:")
print(test["timestamp"].min())
print(test["timestamp"].max())
```

```
Период исходных данных:
2026-01-01 00:16:27
2026-04-10 23:47:44

Период test:
2026-04-11 01:45:36
2026-04-30 22:29:24
```

Колибровочная выборка `calibration` также должна следовать по времени после `train`, а не случайно выбираться из всей истории.

Запишем в `calibration` последние 20% событий.

In [ ]:
```
calibration_start = int(len(data) * 0.80)

train = (data.iloc[:calibration_start]
        .copy().reset_index(drop=True))

calibration = (data.iloc[calibration_start:]
        .copy().reset_index(drop=True))

print("train:")
print("  событий:", len(train))
```

```
print("  атак:", train["is_attack"].sum())
print(
    "  период:",
    train["timestamp"].min(),
    "—",
    train["timestamp"].max())

print("\ncalibration:")
print("  событий:", len(calibration))
print("  атак:", calibration["is_attack"].sum())
print(
    "  период:",
    calibration["timestamp"].min(),
    "—",
    calibration["timestamp"].max())

print("\ntest:")
print("  событий:", len(test))
print("  атак:", test["is_attack"].sum())
print(
    "  период:",
    test["timestamp"].min(),
    "—",
    test["timestamp"].max())
```

```
train:
  событий: 4134
  атак: 231
  период: 2026-01-01 00:16:27 — 2026-03-21 09:02:37

calibration:
  событий: 1034
  атак: 33
  период: 2026-03-21 16:49:28 — 2026-04-10 23:47:44

test:
  событий: 1137
  атак: 61
  период: 2026-04-11 01:45:36 — 2026-04-30 22:29:24
```

Выполняется правильный временной порядок: `train → calibration → test`.

Хронологический Hold-out

Хронологический Hold-out — наиболее простой временной метод.

Выбирается один момент времени t_i относительно которого осуществляется разделение данных на `train` и `validation`:

$$D_{\text{train}} = \{x_i : t_i < T\},$$

$$D_{\text{validation}} = \{x_i : t_i \geq T\},$$

где:

- x_i — i-е событие;
- t_i — время события;
- T — граница разбиения.

Для хронологического Hold-out нельзя использовать `shuffle = True`. Перемешивание нарушит временной порядок.

Реализация.

Разместим последние 20% текущего `train` в `validation`.

In [ ]:
```
validation_start = int(len(train) * 0.80)

train_idx = np.arange(0, validation_start)
validation_idx = np.arange(validation_start, len(train))

X_train = train.iloc[train_idx][FEATURES]
y_train = train.iloc[train_idx]["is_attack"]

X_validation = train.iloc[validation_idx][FEATURES]
y_validation = train.iloc[validation_idx]["is_attack"]

print("train:")
print("  событий:", len(train_idx))
print("  атак:", int(y_train.sum()))
print(
    "  период:",
    train.iloc[train_idx]["timestamp"].min(),
    "—",
    train.iloc[train_idx]["timestamp"].max()
)

print("\nvalidation:")
print("  событий:", len(validation_idx))
print("  атак:", int(y_validation.sum()))
print(
    "  период:",
    train.iloc[validation_idx]["timestamp"].min(),
    "—",
    train.iloc[validation_idx]["timestamp"].max())
```

```
train:
  событий: 3307
  атак: 200
  период: 2026-01-01 00:16:27 — 2026-03-06 07:58:03

validation:
  событий: 827
  атак: 31
  период: 2026-03-06 07:59:03 — 2026-03-21 09:02:37
```

Модель:

- обучается на событиях с 1 января по 6 марта;
- проверяется на событиях с 6 по 21 марта;
- после выбора гиперпараметров окончательно обучается на доступных данных;
- порог настраивается по `calibration`;
- итоговое качество один раз проверяется на `test`.

Ограничения хронологического Hold-out.

- Качество оценивается только на одном временном интервале.
- Результат зависит от выбранной даты разделения.
- В `validation` может оказаться мало атак.
- Некоторые типы атак могут полностью отсутствовать.
- Метод не показывает, как качество изменяется в разные периоды.
- Старые данные могут плохо описывать новое поведение системы.
- Не предотвращается разделение одного инцидента границей времени.
- Не учитывается пересечение временных окон на границе.
- Нельзя применять стратификацию, если она переносит события между датами.

Расширяющееся окно — walk-forward validation

При расширяющемся окне модель несколько раз обучается на прошлом и проверяется на следующем временном периоде.

На каждой итерации `train` становится больше:

Итерация 1: |---- train ----|---------- validation ----------|

Итерация 2: |-------- train --------|------ validation ------|

Итерация 3: |------------ train ------------|-- validation --|

Для итерации j обучающая и валидационная выборки определяются следующим образом:

$$D_{\text{train}}^{(j)} = \{x_i : t_i < T_j\},$$

$$D_{\text{validation}}^{(j)} = \{x_i : T_j \le t_i < T_j + \Delta_v\},$$

где:

- T_j — начало текущего периода валидации;
- Δ_v — длительность периода валидации;
- $D_{\text{train}}^{(j)}$ — множество всех событий, произошедших до момента T_j (обучающая выборка на итерации j);
- $D_{\text{validation}}^{(j)}$ — множество событий, попадающих в следующий временной интервал длиной Δ_v, начинающийся с T_j (валидационная выборка на итерации j).

При таком подходе каждый последующий обучающий набор включает в себя все предыдущие данные — реализуется принцип нарастающего окна:

$$D_{\text{train}}^{(1)} \subset D_{\text{train}}^{(2)} \subset \ldots \subset D_{\text{train}}^{(K)}.$$

Это типичная схема временной кросс-валидации (expanding window), которая особенно актуальна для потоковых данных и задач кибербезопасности: модель последовательно «учится на истории» и проверяется на следующих по времени событиях, что позволяет оценить её устойчивость и способность адаптироваться к эволюции трафика или паттернов атак.

Эту схему также называют:

- forward chaining;
- walk-forward validation.

Для реализации метода можно использовать класс `TimeSeriesSplit` модуля `model_selection` библиотеки `sklearn`.

Основные параметры:

- `n_splits` (по умолчанию: 5): задаёт число разбиений (фолдов);
- `max_train_size` (по умолчанию `None`): ограничивает максимальный размер обучающей выборки (в количестве наблюдений). Зачем нужен: если есть очень длинный ряд и необходимо оценивать модель на «скользящем окне» фиксированной длины (rolling window), а не на нарастающем (expanding window). Пример: если `max_train_size = 1000`, то даже на последнем сплите в `train` будет не больше 1000 последних доступных перед `validation` наблюдений. Это превращает схему из «нарастающего окна» (expanding window) в «скользящее окно» (rolling window);
- `test_size` (по умолчанию автоматически определяется как n//(n_splits+1)): фиксирует размер тестовой выборки (в числе наблюдений). Зачем нужен: когда важно, чтобы во всех сплитах `validation` был одинакового размера (например, горизонт прогноза 30 дней).
- `gap` (по умолчанию 0): добавляет «пропуск» между `train` и `validation`: после `train` пропускается `gap` наблюдений, и только потом начинается `validation`. Зачем нужен: в реальных сценариях бывает полезно исключить ближайшие к `train` точки из `validation`, чтобы убрать эффект «слишком похожих» соседей или учесть задержки в данных/отклике системы.

Хотя параметр называется `test_size`, полученные блоки внутри кросс-валидации используются как `validation`. Настоящий `test` остаётся полностью отложенным.

Реализация через `TimeSeriesSplit`.

Используем пять последовательных `validation` по 500 событий.

In [ ]:
```
X = train[FEATURES]
y = train["is_attack"]

expanding_splitter = TimeSeriesSplit(n_splits = 5,
    # Размер каждого validation в событиях.
    test_size = 500,

    # None означает, что train постоянно расширяется.
    max_train_size = None,

    # В этом блоке зазор пока не используется.
    gap = 0)

expanding_rows = []

for iteration, (train_idx, validation_idx) in enumerate(
    expanding_splitter.split(X), start=1):
    y_train = y.iloc[train_idx]
    y_validation = y.iloc[validation_idx]

    expanding_rows.append({
        "iteration": iteration,
        "train_size": len(train_idx),
        "train_start":
            train.iloc[train_idx]["timestamp"].min(),
        "train_end":
            train.iloc[train_idx]["timestamp"].max(),
        "train_attacks": int(y_train.sum()),
        "validation_size": len(validation_idx),
        "validation_start":
            train.iloc[validation_idx]["timestamp"].min(),
        "validation_end":
            train.iloc[validation_idx]["timestamp"].max(),
        "validation_attacks":
            int(y_validation.sum()),
        "validation_attack_share":
            y_validation.mean()})

expanding_result = pd.DataFrame(expanding_rows)

expanding_result[["iteration",
                  "train_size",
                  "train_attacks",
                  "validation_size",
                  "validation_attacks",
                  "validation_attack_share"]]
```

Out[ ]:

	iteration	train_size	train_attacks	validation_size	validation_attacks	validation_attacl
0	1	1634	92	500	26	
1	2	2134	118	500	43	
2	3	2634	161	500	23	
3	4	3134	184	500	28	
4	5	3634	212	500	19	

Обратите внимание: доля атак изменяется от 3,8 до 8,6%. Это не ошибка разбиения. В реальной сети частота атак изменяется во времени.

Временная кросс-валидация не должна искусственно переносить атаки между периодами только ради одинаковых долей классов.

Первый `train` размером 1634 события определяется автоматически из параметров:

```
expanding_splitter = TimeSeriesSplit(n_splits = 5, test_size = 500)
```

Общий размер данных: n = 4134.

Для пяти итераций требуется пять последовательных блоков `validation` по 500 событий: 5 · 500 = 2500.

Оставшиеся начальные события образуют первый `train` : 4134 - 2500 = 1634.

Схематично:

```
|-- 1634 --|-- 500 --|-- 500 --|-- 500 --|-- 500 --|-- 500 --|
 начальный    val 1     val 2     val 3     val 4     val 5
   train
```

Поскольку используется расширяющееся окно, после каждой итерации предыдущий `validation` присоединяется к следующему `train` :

Важная особенность покрытия.

В обычном K-Fold каждый объект один раз становится валидационным.

В TimeSeriesSplit самые ранние события нужны для формирования начального `train` , поэтому они могут ни разу не попасть в `validation` .

В нашем примере:

- первые 1643 событий формируют начальную историю;
- последние 2500 событий последовательно используются в `validation` ;
- каждое из этих 2500 событий становится валидационным один раз.

Это нормальное свойство walk-forward validation. Нельзя проверять первую модель без хотя бы некоторого объёма предшествующих данных.

Ограничения расширяющегося окна.

- Старые данные остаются в `train` на всех последующих итерациях.
- Устаревшее поведение может ухудшать модель при дрейфе данных.
- Первые события никогда не используются как `validation`.
- Начальные итерации обучаются на меньшем объёме данных.
- Количество атак в разных периодах может сильно отличаться.
- Отдельные сценарии атак могут отсутствовать.
- TimeSeriesSplit разделяет строки, а не реальные календарные интервалы.
- Не учитываются группы связанных событий.
- Один инцидент может пересечь границу `train` и `validation`.

Скользящее окно

При скользящем окне размер `train` ограничен. На каждой итерации модель обучается только на последних событиях перед `validation`.

```
Итерация 1: |---- train ----|-- validation --|
Итерация 2:      |---- train ----|-- validation --|
Итерация 3:           |---- train ----|-- validation --|
```

Для итерации j обучающая и валидационная выборки формируются по временным окнам:

$$D_{\text{train}}^{(j)} = \{x_i : T_j - \Delta_t \le t_i < T_j\},$$

$$D_{\text{validation}}^{(j)} = \{x_i : T_j \le t_i < T_j + \Delta_v\},$$

где:

- T_j — начало периода валидации на итерации j;
- Δ_t — длительность временного окна для обучающей выборки;
- Δ_v — длительность периода валидации.

Такая схема соответствует подходу со скользящим окном: на каждой итерации модель обучается на предшествующем интервале длиной Δ_t, а проверяется на следующем за ним интервале длиной Δ_v. Это особенно удобно при анализе временных данных в кибербезопасности (например, сетевого трафика или логов событий), где важно учитывать эволюцию паттернов во времени и избегать утечки информации из «будущего» в обучение.

В отличие от расширяющегося окна, слишком старые события удаляются из `train`.

Это полезно, если:

- нормальное поведение сети быстро меняется;
- появляются новые сервисы;

- меняются профили пользователей;
- происходит обновление инфраструктуры;
- старые атаки теряют актуальность.

Реализация через `TimeSeriesSplit`.

Ограничим `train` последними 1500 событиями.

In [ ]:
```
sliding_splitter = TimeSeriesSplit(n_splits = 5,
    # 500 следующих событий используются как validation.
    test_size = 500,

    # В train остаются только последние 1500 событий.
    max_train_size = 1500,

    gap = 0)

sliding_rows = []

for iteration, (train_idx, validation_idx) in enumerate(
    sliding_splitter.split(X), start=1):
    y_train = y.iloc[train_idx]
    y_validation = y.iloc[validation_idx]

    sliding_rows.append({
        "iteration": iteration,
        "train_size": len(train_idx),
        "train_start":
            train.iloc[train_idx]["timestamp"].min(),
        "train_end":
            train.iloc[train_idx]["timestamp"].max(),
        "train_attacks": int(y_train.sum()),
        "validation_size": len(validation_idx),
        "validation_start":
            train.iloc[validation_idx]["timestamp"].min(),
        "validation_end":
            train.iloc[validation_idx]["timestamp"].max(),
        "validation_attacks":
            int(y_validation.sum()),
        "validation_attack_share":
            y_validation.mean()})

sliding_result = pd.DataFrame(sliding_rows)

sliding_result[["iteration",
                "train_size",
                "train_attacks",
                "validation_size",
                "validation_attacks",
                "validation_attack_share"]]
```

Out[ ]:

	iteration	train_size	train_attacks	validation_size	validation_attacks	validation_attacl
0	1	1500	92	500	26	
1	2	1500	85	500	43	
2	3	1500	104	500	23	
3	4	1500	92	500	28	
4	5	1500	94	500	19	

Ограничения скользящего окна.

- Необходимо выбирать размер окна `train`.
- Слишком короткое окно содержит мало примеров атак.
- Слишком длинное окно начинает напоминать расширяющуюся схему.
- Старые, но полезные сценарии полностью удаляются.
- Редкие виды атак могут исчезнуть из `train`.
- Меньший объём данных повышает нестабильность модели.
- Размер в событиях не соответствует фиксированной календарной длительности.
- Не учитываются группы инцидентов и сессий.
- Граница окна может разрезать один инцидент.

Блочная временная валидация

Временная шкала разделяется на несколько непересекающихся циклов. Каждый цикл содержит собственный период `train`, после которого расположен период `validation`.

Схема для трех итераций:

```
Время →

|- train 1 -|- validation 1 -|
                              |- train 2 -|- validation 2 -|
                                                           |-
train 3 -|- validation 3 -|
```

В отличие от расширяющегося окна:

- следующий `train` не включает всю предыдущую историю;
- объём обучающих данных не увеличивается;
- события разных итераций не пересекаются.

В отличие от скользящего окна:

- соседние циклы не перекрываются;
- одно событие не используется в нескольких итерациях;
- после окончания `validation` начинается новый независимый цикл.

В задачах кибербезопасности такая схема может моделировать, например:

- обучение модели на первой части месяца и проверку на второй;
- обучение на рабочих днях и проверку на следующих выходных;
- сравнение качества модели в нескольких независимых временных периодах;
- периодическое переобучение только на данных последнего отчётного периода.

Пример.

Предположим, система мониторинга работает в течение 80 дней. Разделим этот период на четыре цикла по 20 дней:

- первые 11 дней — `train`;
- следующие 9 дней — `validation`.

Получаем:

Итерация	Train	Validation
1	дни 1–11	дни 12–20
2	дни 21–31	дни 32–40
3	дни 41–51	дни 52–60
4	дни 61–71	дни 72–80

На каждой итерации модель обучается только на событиях соответствующего локального периода.

Например:

- первая модель изучает сетевое поведение начала января;
- затем проверяется на событиях второй половины январского периода;
- вторая модель отдельно обучается на следующем временном блоке;
- события первой итерации во второй итерации больше не используются.

In [ ]:
```
FEATURES = ["connections_per_min",
            "unique_destinations",
            "outbound_mb",
            "failed_logins",
            "duration_sec"]

# test содержит самый поздний период.
test = (df.loc[df["dataset_part"] == "final_test"]
      .sort_values("timestamp").reset_index(drop=True))

# События, доступные для разработки модели.
data = (df.loc[df["dataset_part"] == "development"]
      .sort_values("timestamp").reset_index(drop=True))

# Последние 20% временно упорядоченных событий
# оставляем для calibration.
calibration_start = int(len(data) * 0.80)
```

```
train = (data.iloc[:calibration_start].copy().reset_index(drop=True))
calibration = (data.iloc[calibration_start:].copy().reset_index(drop=True))

print("Размер train:", len(train))
print("Период train:",train["timestamp"].min(),"—",train["timestamp"].max())
```

```
Размер train: 4134
Период train: 2026-01-01 00:16:27 — 2026-03-21 09:02:37
```

Реализация генератора блочных разбиений.

In [ ]:

```
def blocked_time_splits(data,
                        n_splits=4,
                        train_days=11,
                        validation_days=9,
                        time_column="timestamp"):
    """
    Формирует непересекающиеся временные разбиения.

    Каждый цикл содержит:
        train -> validation

    Параметры
    ----------
    data: Таблица с событиями, отсортированными по времени.
    n_splits: Количество временных циклов.
    train_days: Продолжительность train в днях.
    validation_days: Продолжительность validation в днях.
    time_column: Название столбца с временем события.

    Возвращает
    ----------
    train_idx: Индексы событий текущего train.
    validation_idx: Индексы событий текущего validation.
    """
    # Вычисляем длительность одного полного цикла: train + validation.
    # Это нужно, чтобы сдвигать начало каждого следующего цикла.
    cycle_duration = pd.Timedelta(days=train_days + validation_days)

    # Длительность окна обучения (в днях).
    train_duration = pd.Timedelta(days=train_days)

    # Длительность окна валидации (в днях).
    validation_duration = pd.Timedelta(days=validation_days)

    # Находим минимальное время в данных — это старт первого цикла.
    first_cycle_start = data[time_column].min()

    # Проходим по всем требуемым итерациям (разбиениям).
    for iteration in range(n_splits):

        # Вычисляем начало текущего цикла:
        # сдвигаем старт на количество полных циклов, прошедших до этой итерации
        cycle_start = (first_cycle_start + iteration * cycle_duration)

        # Определяем границы обучающего окна:
        # train начинается в начале цикла и длится train_days.
        train_start = cycle_start
        train_end = train_start + train_duration
```

```
        # Границы валидационного окна:
        # validation начинается сразу после train и длится validation_days.
        validation_start = train_end
        validation_end = (validation_start + validation_duration)

        # Создаём булеву маску для событий, попадающих в train-окно.
        # Используем полуоткрытый интервал [start, end), чтобы избежать дублиров
        train_mask = ((data[time_column] >= train_start)
                      & (data[time_column] < train_end))

        # Аналогично создаём маску для validation-окна.
        validation_mask = ((data[time_column] >= validation_start)
                           & (data[time_column] < validation_end))

        # Получаем индексы строк, удовлетворяющих маске.
        # np.flatnonzero возвращает массив номеров строк (позиций), где маска Tr
        train_idx = np.flatnonzero(train_mask)
        validation_idx = np.flatnonzero(validation_mask)

        # Возвращаем индексы для текущей итерации через yield:
        # функция становится генератором и отдаёт пары (train_idx, validation_id
        yield train_idx, validation_idx
```

Применение созданного генератора.

In [ ]:

```
blocked_rows = []

for iteration, (train_idx, validation_idx) in enumerate(
    blocked_time_splits(train,
                        n_splits=4,
                        train_days=11,
                        validation_days=9), start=1):
    y_train = train.iloc[train_idx]["is_attack"]
    y_validation = train.iloc[validation_idx]["is_attack"]

    blocked_rows.append({"iteration": iteration,
                         "train_start": train.iloc[train_idx]["timestamp"].min()
                         "train_end": train.iloc[train_idx]["timestamp"].max(),
                         "train_size": len(train_idx),
                         "train_attacks": int(y_train.sum()),
                         "validation_start": train.iloc[validation_idx]["timesta
                         "validation_end": train.iloc[validation_idx]["timestamp
                         "validation_size": len(validation_idx),
                         "validation_attacks": int(y_validation.sum()),
                         "validation_attack_share": y_validation.mean()})

blocked_result = pd.DataFrame(blocked_rows)

blocked_result[[
            "iteration",
            "train_size",
            "train_attacks",
            "validation_size",
            "validation_attacks",
            "validation_attack_share"]]
```

Размеры различаются, хотя длительность периодов одинакова. Причина в том, что интенсивность поступления событий меняется во времени.

Временное разбиение не гарантирует присутствия всех типов атак.

In [ ]:
```
for iteration, (train_idx, validation_idx) in enumerate(
    blocked_time_splits(train,
                        n_splits=4,
                        train_days=11,
                        validation_days=9), start=1):
    scenarios = (train.iloc[validation_idx]. \
                 loc[train.iloc[validation_idx]["is_attack"] == 1, "scenario"].v

    print(f"\nИтерация {iteration}:")
    print(scenarios)
```

```
Итерация 1:
scenario
exfiltration    7
brute_force     5
beaconing       5
Name: count, dtype: int64

Итерация 2:
scenario
brute_force     8
exfiltration    8
beaconing       7
Name: count, dtype: int64

Итерация 3:
scenario
brute_force     8
beaconing       6
exfiltration    5
Name: count, dtype: int64

Итерация 4:
scenario
beaconing       8
exfiltration    6
brute_force     5
Name: count, dtype: int64
```

В рассматриваемых блоках `validation` присутствуют не все типы атак. Например, все периоды не содержат:

- `port_scan`;
- `lateral_movement`.

Это не ошибка метода. Такие атаки действительно не происходили в соответствующих временных интервалах.

Нельзя переносить их из других дат только для искусственного выравнивания классов — это нарушит временную причинность.

Если отдельные блоки содержат слишком мало атак, можно:

- увеличить продолжительность `validation`;
- использовать меньше временных циклов;

- объединить результаты нескольких `validation`.

Литература

1. Чио К., Фримэн Д. Машинное обучение и безопасность [Электронный ресурс]:. - Москва: ДМК Пресс, 2020. - 388 с. – Режим доступа: https://e.lanbook.com/book/131707.
2. Протодьяконов А. В., Пылов П. А., Садовников В. Е. Алгоритмы Data Science и их практическая реализация на Python [Электронный ресурс]:учебное пособие. - Вологда: Инфра-Инженерия, 2022. - 392 с. – Режим доступа: https://e.lanbook.com/book/281189.
3. Криволапов С.Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python [Электронный ресурс]:Учебное пособие. - Москва: ООО "Научно-издательский центр ИНФРА-М", 2025. - 678 с. – Режим доступа: https://znanium.ru/catalog/document?id=450872.
4. Коротеев М.В. Основы машинного обучения на Python [Электронный ресурс]:Учебник. - Москва: КноРус, 2025. - 431 с. – Режим доступа: https://book.ru/book/957785.
5. Груздев А. В. План, примеры и метрики качества [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 814 с. – Режим доступа: https://e.lanbook.com/book/314948.
6. Груздев А. В. Инструменты и валидация [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 816 с. – Режим доступа: https://e.lanbook.com/book/314945.